TOPIC 3

Digital Logic Design Principles

Subtopic 3a: Boolean algebra and logic gates · 3b: Combinational logic design · 3c: Sequential logic — flip-flops, registers and state machines

- 3a · Binary, Boolean algebra, the seven gates, De Morgan, SOP/POS, K-map minimisation
- 3b · Design procedure, adders, MUX, decoders, ALU, hazards, delay and fan-out
- 3c · Latches, flip-flops, timing, metastability, registers, counters, FSMs
- Tools · Logisim-Evolution, Icarus Verilog, GTKWave, Yosys · Tutorials T1–T4

TOPIC 3 · ROADMAP

What This Topic Covers and How the 4 Hours Are Spent

Where this sits. Topic 2 gave you the RTL methodology — the idea that we describe hardware, not program it. Topic 3 gives you the **hardware vocabulary itself**. Every Verilog construct you write in Topic 4 maps onto one of the structures in this topic. Without this, RTL coding is guesswork.

Syllabus bullet (NIE/ELE/N0102, Topic 3 — 4 hours)	Covered by	Time
Boolean algebra and logic gates	**3a — slides 4–18**	1 h 15 m
Combinational logic design concepts	**3b — slides 19–32**	1 h 15 m
Sequential logic design concepts	**3c — slides 33–45**	45 m
Flip-flops, registers and state machines	**3c — slides 33–59**	45 m
Hands-on tools and tutorials T1–T4	**slides 60–66 + workbook**	lab time

Learning outcomes — by the end of this topic you can

· Simplify any Boolean function algebraically and with a Karnaugh map, and justify why the minimal form matters in silicon.
· Design, analyse and hand-verify adders, multiplexers, decoders, comparators and an ALU.
· Explain latches vs flip-flops, compute f_{max} from a timing path, and recognise setup/hold and metastability failures.
· Design a finite state machine end to end — state diagram → table → encoding → logic → Verilog → simulation → synthesis.

How to run the session

Deliver 3a and 3b before the break, with a 10-minute Logisim demo after each; run 3c after the break and finish in the lab with tutorials T1–T4.
The six worked numerical examples are your checkpoints — if the room cannot follow one, slow down rather than skipping ahead.

TOPIC 3 · MOTIVATION

Why a Verilog Engineer Still Needs Gate-Level Thinking

A synthesiser turns your Verilog into gates — so why learn gates at all? Because the tool optimises what you WROTE, not what you MEANT. Every one of the following is a real, common bug that is invisible at the Verilog level and obvious at the logic level.

You write	Tool infers	Consequence	You needed to know
an **incomplete if/else** in a combinational block	a **transparent LATCH** instead of pure gates	timing cannot be **closed; chip fails** intermittently	**what a latch IS** and why level-sensitivity is deadly

Three more examples of the same pattern

Async reset written as sync → the design never leaves an unknown state after power-up. You needed flip-flop reset behaviour.

A signal crossed from another clock → random one-in-a-million failures. You needed metastability.

A 32-bit ripple adder in a 1 GHz design → timing fails by 4 ns. You needed carry propagation delay.

The rule for this whole module: you must be able to picture the hardware your code will become — BEFORE you run synthesis.

SUBTOPIC 3A

Boolean Algebra and Logic Gates

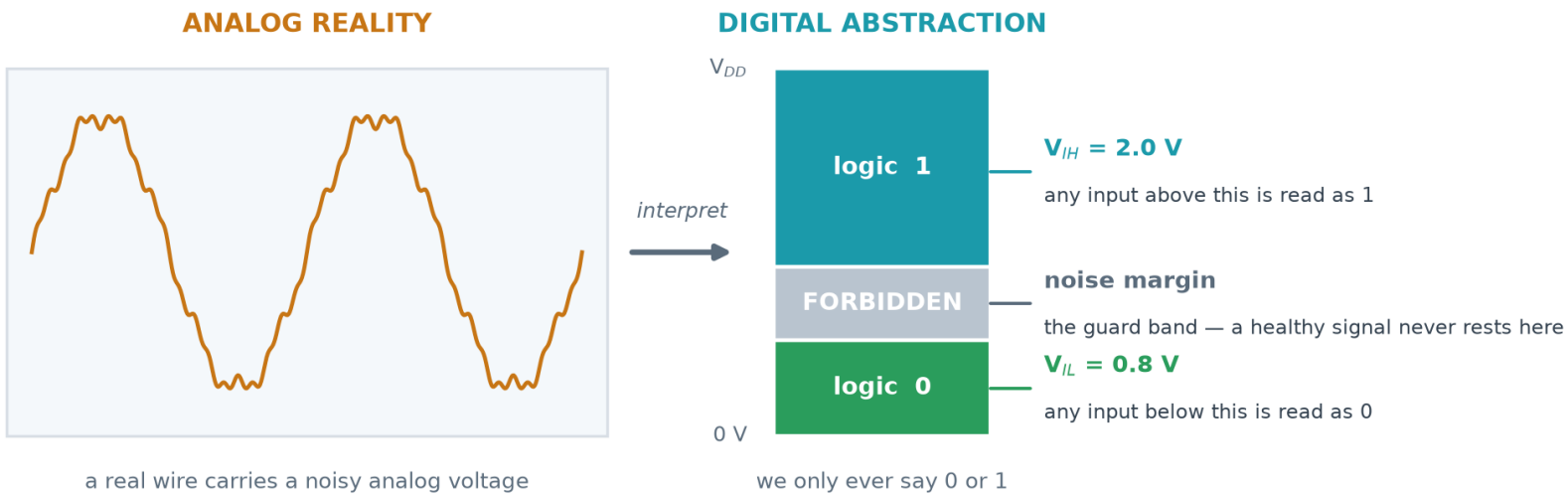
The mathematics of 0 and 1 — and the physical devices that implement it.

- Binary, the digital abstraction, number systems and codes
- The seven gates, universal gates, and De Morgan's theorems
- Boolean axioms, laws and theorems — with proofs you can do on paper
- Canonical forms (SOP / POS) and Karnaugh-map minimisation

Why Digital? The Two-Valued Abstraction

A digital circuit is an analog circuit that we have agreed to misread. The wire really does carry a continuous voltage. We simply declare two bands — anything below V^{I}_{L} counts as 0, anything above V^{I}_{H} counts as 1 — and design so a signal never rests in between. That single decision buys us noise immunity, testability, and the ability to reason with algebra instead of calculus.

The digital abstraction — a continuous voltage read as one of two symbols



Why it matters: the guard band makes digital logic RESTORATIVE — noise, ageing and temperature drift are discarded at every gate, so a signal can cross a whole chip without degrading.

Terms you must be able to define

Bit — one binary digit. **Logic level** — the 0/1 interpretation of a voltage. **Noise margin** — how much interference a signal can absorb and still be read correctly ($NM^{L} = V^{I}_{L} - V^{O}_{L}$, $NM^{H} = V^{O}_{H} - V^{I}_{H}$). **Restoring logic** — every gate outputs a clean full-swing level, so noise never accumulates.

TOPIC 3A · REPRESENTATION

Number Systems and Codes You Will Meet in RTL

Hardware only stores bits. What those bits MEAN is a convention you choose and must apply consistently — the same 8 bits are 180 unsigned, −76 signed, 0xB4 in hex, or a BCD digit pair. Verilog will happily let you mix them up; the bug appears in silicon.

One number, four costumes — and how negative numbers are stored

BINARY

1011 0100

base 2 — what the wires carry

HEX

0xB4

base 16 — one hex digit = 4 bits

DECIMAL

180

base 10 — what humans read

BCD

0001 1000 0000

each decimal digit in its own nibble

TWO'S COMPLEMENT — how −5 is stored in 8 bits

MSB = sign bit · n bits cover -2^{n-1} … $+2^{n-1}-1$

`0000 0101` → `1111 1010` → `1111 1011` → `1111 1011`

start with +5 · invert every bit · add 1 · this IS −5

Gray code

Successive values differ in exactly ONE bit.

Used for K-maps, FIFO pointers crossing clock domains, and shaft encoders — because a multi-bit change can be sampled mid-flight.

BCD

Each decimal digit in its own 4-bit nibble.

Wastes 6 of 16 codes but makes decimal display and financial arithmetic exact. The mod-10 counter you build later is a BCD counter.

Two's complement

The universal signed format.

One adder handles + and −; there is only one zero. In Verilog, declare `signed` or the tool assumes unsigned and your comparisons invert.

TOPIC 3A · THE ALGEBRA

Boolean Variables, Operators and Truth Tables

Boolean algebra (George Boole, 1854; applied to switching circuits by Claude Shannon, 1937) **is an algebra over exactly two values.** A variable is 0 or 1. There are three primitive operators, and every digital circuit that has ever been built is a composition of these three.

Operator	Symbols used	Reads as	Result is 1 when…
AND	A · B A ∧ B AB A & B	conjunction	BOTH inputs are 1
OR	A + B A ∨ B A | B	disjunction	AT LEAST ONE input is 1
NOT	A' Ā ¬A ~A	complement / negation	the input is 0

The truth table — the ground truth of every combinational function

A truth table lists the output for every possible input combination. For n inputs there are **2^n rows** — 2 inputs → 4 rows, 4 inputs → 16 rows, 16 inputs → 65 536 rows. This exponential growth is exactly why we need algebra and K-maps rather than exhaustive tables.

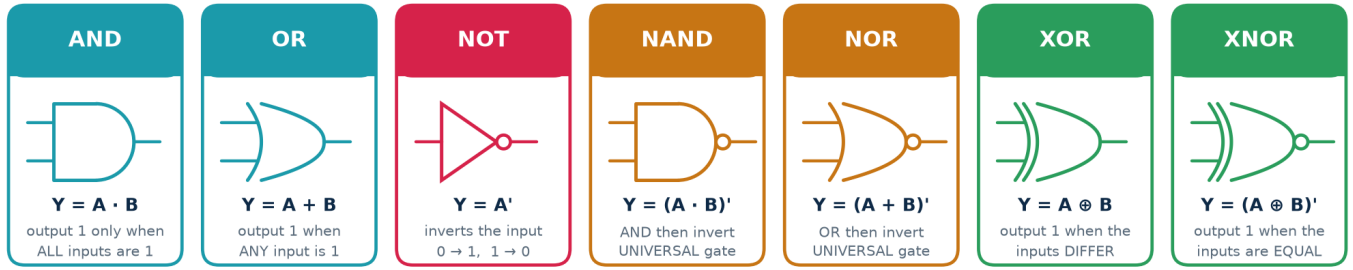
Two circuits are **functionally equivalent** if and only if their truth tables match on every row — no matter how differently they are drawn. Equivalence checking (a signoff step in Topic 6) automates exactly this.

Precedence: NOT binds tightest, then AND, then OR — so A + B·C means A + (B·C). When in doubt, bracket it. Verilog follows the same order.

The Seven Logic Gates — Symbol, Expression, Truth Table

A logic gate is a physical circuit (in CMOS, a handful of transistors) that computes one Boolean operator. Learn to read these symbols instantly — schematics, synthesis reports and datasheets all assume you can.

The seven logic gates — symbol, Boolean expression, truth table



Combined truth table — read one column to get that gate's behaviour

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	**0**	0	0	1	1	0	1
0	**1**	0	1	1	0	1	0
1	**0**	0	1	1	0	1	0
1	**1**	1	1	0	0	0	1

NOT is single-input: A = 0 → Y = 1, A = 1 → Y = 0 · '·' means AND, '+' means OR, an apostrophe (or overbar) means NOT

Transistor cost — why NAND and NOR are the cheapest gates in CMOS

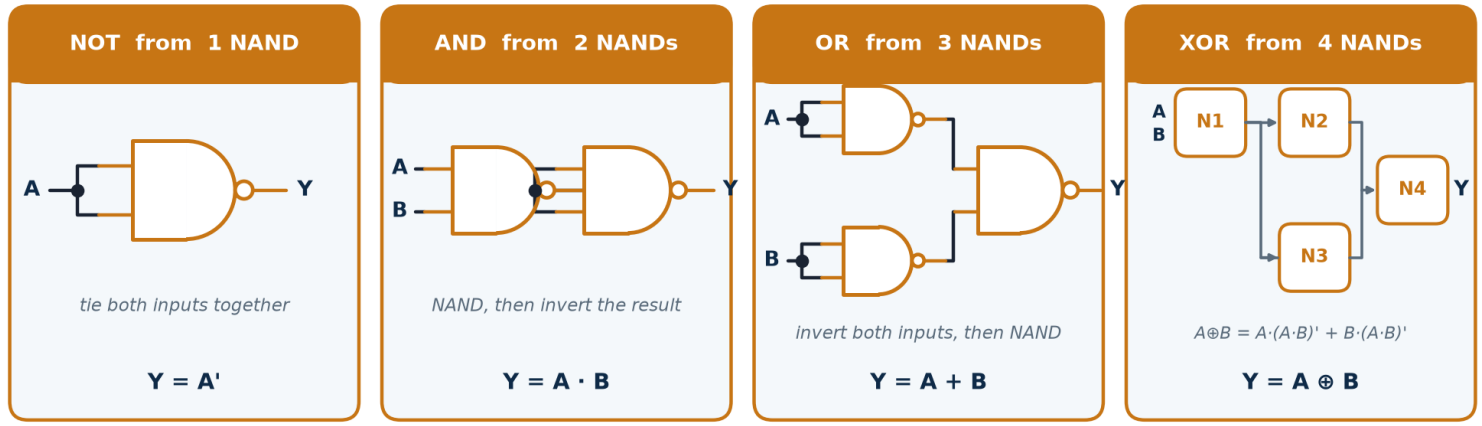
In static CMOS, an inverter costs 2 transistors, a 2-input NAND or NOR costs 4, but a 2-input AND or OR costs 6 *(they are built as NAND/NOR followed by an inverter)*. XOR costs 8–12. This is why standard-cell libraries are dominated by NAND/NOR/inverter cells and why synthesis output rarely contains a literal AND gate.

TOPIC 3A · UNIVERSALITY

Universal Gates — NAND and NOR Can Build Anything

A gate is UNIVERSAL if any Boolean function can be built from copies of it alone. NAND and NOR are the only two-input gates with this property (AND, OR and XOR are not — none of them can produce a NOT). The proof is constructive: build NOT, AND and OR from it, and since every function has an SOP form, you are done.

NAND is universal — every other gate can be built from NAND alone



Why industry cares: a foundry only has to characterise, optimise and yield a SMALL set of primitive cells. NOR is universal too — the same proof runs by duality.

Why it matters in manufacturing

One well-characterised cell can be tiled everywhere, so the foundry optimises, models and yields a small library instead of a large one.

Why it matters in design

Synthesis 'technology maps' your logic onto whatever cells the library actually has — so your AND gate may well appear as two NANDs in the netlist.

Boolean Axioms, Laws and Theorems

Every one of these can be proved with a truth table, and every one is a legal, behaviour-preserving circuit transformation.
Synthesis tools apply them thousands of times per second; you apply them by hand to check the tool and to simplify before you write the code.

Law	OR form	AND form
Identity	A + 0 = A	A · 1 = A
Null / Dominance	A + 1 = 1	A · 0 = 0
Idempotent	A + A = A	A · A = A
Complement	A + A' = 1	A · A' = 0
Involution	(A')' = A	—
Commutative	A + B = B + A	A · B = B · A
Associative	A + (B + C) = (A + B) + C	A · (B · C) = (A · B) · C
Distributive	A · (B + C) = A·B + A·C	A + (B · C) = (A+B) · (A+C)
Absorption	A + A·B = A	A · (A + B) = A
Simplification	A + A'·B = A + B	A · (A' + B) = A · B
Consensus	AB + A'C + BC = AB + A'C	(A+B)(A'+C)(B+C) = (A+B)(A'+C)

Two ideas that make the table half as long

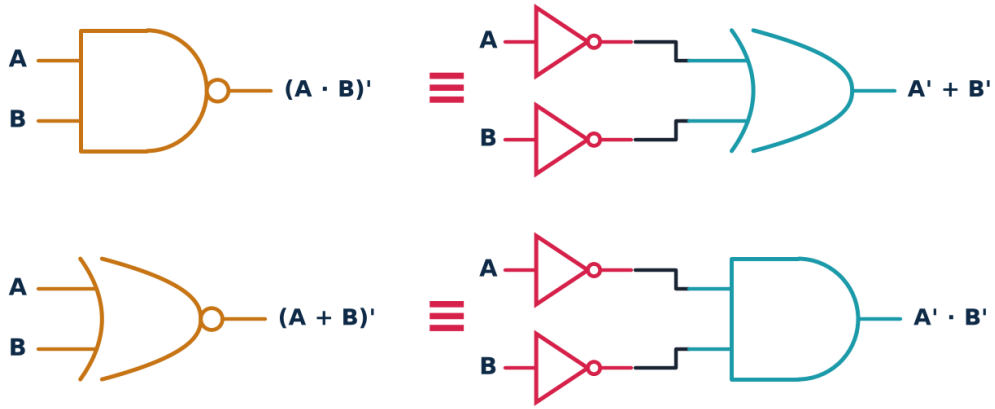
Duality. Swap every AND ↔ OR and every 0 ↔ 1 and a true statement stays true — which is why the two columns above are mirror images. You only have to memorise one column.

Why the second distributive law surprises people. A + B·C = (A+B)(A+C) has no counterpart in ordinary arithmetic. Verify it on all 8 rows once and you will trust it forever.

De Morgan's Theorems — The Most-Used Identity in RTL

Break the bar, change the operator. These two identities let you push inversions through a network, convert any AND-OR structure into an all-NAND structure, and read active-low signals correctly. In practice this is the theorem you will use most often at a whiteboard.

De Morgan's theorems — "break the bar, change the operator"



$$(A \cdot B)' = A' + B'$$

$$(A + B)' = A' \cdot B'$$

Read it as: a bubble on the output can be pushed back to bubbles on every input, provided you swap AND ↔ OR. This is why NAND/NOR logic is so flexible in real gate libraries.

Proof by perfect induction (exhaustive truth table)

```
A=0 B=0 → (A·B)' = 1, A'+B' = 1+1 = 1  ✓        A=0 B=1 → (A·B)' = 1, A'+B' = 1+0 = 1  ✓
A=1 B=0 → (A·B)' = 1, A'+B' = 0+1 = 1  ✓        A=1 B=1 → (A·B)' = 0, A'+B' = 0+0 = 0  ✓
```

All four rows agree, so the two expressions are the same function. The theorem generalises to n variables: (ABC…)' = A'+B'+C'+…

Active-low reading: a signal named `rst_n` asserted low means 'reset when rst_n = 0'. De Morgan is how you convert a spec written in active-low language into positive logic without errors.

TOPIC 3A · TIERED DEPTH

Understanding Boolean Algebra at Four Levels

The same idea, deepened. Use the level that matches your audience — and make sure you can climb all four.

BASIC	Circuits work with just two values, 0 and 1. Three operations — AND, OR and NOT — combine them, and a truth table lists what comes out for every possible input.
INTERMEDIATE	Boolean algebra is a formal algebra with its own laws. Those laws let you rewrite an expression into an equivalent one that needs fewer gates — which is exactly what minimisation means.
ADVANCED	Any function has canonical SOP and POS forms derived directly from its truth table. Minimisation (K-map, Quine–McCluskey, Espresso) finds a cover of prime implicants that is cheapest under a chosen cost model — literals, gates, or area.
INDUSTRY	Nobody minimises by hand at scale. Synthesis uses BDDs, AIGs and SAT-based rewriting (ABC) against a real standard-cell library, optimising area, timing and power together. Your job is to write clean RTL, set correct constraints, and read the report critically.

Worked: Algebraic Simplification, Step by Step

Problem. Simplify F = A·B·C + A·B·C' + A·B'·C + A'·B·C and state how many gates you saved.

Each line names the law that justifies it — never skip that column

```
F = ABC + ABC' + AB'C + A'BC

  = AB(C + C')  + AB'C + A'BC        group terms 1 and 2  (distributive)
  = AB(1)       + AB'C + A'BC        complement:  C + C' = 1
  = AB          + AB'C + A'BC        identity:    X · 1  = X

  = A(B + B'C)  + A'BC               factor A out of terms 1 and 2
  = A(B + C)    + A'BC               simplification:  B + B'C = B + C
  = AB + AC     + A'BC               expand

  = AB + A'BC   + AC                 reorder
  = B(A + A'C)  + AC                 factor B
  = B(A + C)    + AC                 simplification again
  = AB + BC     + AC

F = AB + BC + AC        ←  the 'majority' function: 1 when ≥ 2 inputs are 1
```

Cost before

4 AND gates (3-input)

1 OR gate (4-input)

3 inverters · 12 literals

Cost after

3 AND gates (2-input)

1 OR gate (3-input)

0 inverters · 6 literals

Sanity check

Build the truth table for both forms and compare all 8 rows. Never trust an algebraic simplification you have not checked.

SOP and POS — Reading Algebra Straight Off a Truth Table

Given any truth table you can write down two exact expressions immediately, with no thinking required. They are called canonical because they are unique. They are also usually far from minimal — which is what the next two slides fix.

Canonical forms — every truth table has two exact algebraic twins

A	B	C	Y	minterm (Y = 1)	maxterm (Y = 0)
0	0	0	**0**	m_0 = A'B'C'	M_0 = A + B + C
0	0	1	**1**	m_1 = A'B'C	M_1
0	1	0	**0**	m_2	M_2 = A + B' + C
0	1	1	**1**	m_3 = A'BC	M_3
1	0	0	**0**	m_4	M_4 = A' + B + C
1	0	1	**1**	m_5 = AB'C	M_5
1	1	0	**0**	m_6	M_6 = A' + B' + C
1	1	1	**1**	m_7 = ABC	M_7

SUM OF PRODUCTS (SOP)

Y = Σm(1, 3, 5, 7)

= A'B'C + A'BC + AB'C + ABC
OR together one product term per Y=1 row

PRODUCT OF SUMS (POS)

Y = ΠM(0, 2, 4, 6)

= (A+B+C)(A+B'+C)(A'+B+C)(A'+B'+C)
AND together one sum term per Y=0 row

Both describe the SAME function — here both reduce to just Y = C.
Canonical ≠ minimal: that is the job of K-maps.

Minterm m_i

The AND term that is 1 for exactly ONE input combination. Write the variable plain if its bit is 1, complemented if 0.

Maxterm M_i

The OR term that is 0 for exactly ONE input combination — the complement rule is reversed.

Relationship

$M_i = (m_i)'$. A function's SOP minterm list and POS maxterm list are complementary sets of the same 2^n indices.

Karnaugh Maps — Minimisation You Can Do by Eye

A K-map is a truth table redrawn so that physically adjacent cells differ in exactly one variable. That turns the algebraic law X·Y + X·Y' = X into a visual one: any group of adjacent 1s collapses into a single, shorter product term. Practical up to 4 variables (5–6 with effort).

Karnaugh maps — adjacency made visible by Gray-code ordering

2 vars · 4 cells

3 vars · 8 cells · group of 4 → Y = B

4 vars · 16 cells · edges wrap

Why Gray code?

00 → 01 → 11 → 10

Only ONE bit changes between neighbours, so two touching cells always differ in exactly one variable — which is exactly the condition for XY + XY' = X.

The five rules of grouping

1. Only ADJACENT cells may be grouped — Gray ordering guarantees neighbours differ in one variable.
2. Group sizes must be powers of two: 1, 2, 4, 8, 16.
3. Make each group as LARGE as possible.
4. Use as FEW groups as possible — but every 1 must be covered at least once (overlap is allowed).
5. The map WRAPS: left edge touches right edge, top touches bottom, and the four corners are adjacent.

Don't-cares (X) may be absorbed into a group when they make it bigger — but never grouped on their own.

Vocabulary: an **implicant** is any legal group; a **prime implicant** is one that cannot be made larger; an **essential prime implicant** is the only group covering some particular 1 — it must appear in every minimal solution.

TOPIC 3A · WORKED EXAMPLE

A 4-Variable K-Map with Don't-Cares

Don't-cares (X) are input combinations that can never occur, or whose output nobody reads. You may assign each one 0 or 1 — whichever makes your groups bigger. They are free optimisation, and forgetting to declare them is a common source of oversized logic.

Worked K-map: F(A,B,C,D) = Σm(0,1,2,5,8,9,10) + d(3,7)

Reading the groups

- **B' C'** — m0, m1, m8, m9 — wraps top ↔ bottom edge
- **B' D'** — m0, m2, m8, m10 — the four corners
- **A' D** — m1, m3, m5, m7 — the X cells grow it to 4

F = B'C' + B'D' + A'D

3 product terms · 6 literals
canonical SOP would need 7 terms / 28 literals

Every '1' must be covered at least once. m5 is covered ONLY by A'D, which makes A'D an essential prime implicant. The X cells were used to enlarge groups, never covered for their own sake.

The procedure, in the order you should always follow it

1. Fill the map from the truth table. 2. Circle every ESSENTIAL prime implicant first. 3. Cover the remaining 1s with the fewest, largest groups. 4. Read each group as a product term: keep the variables that stay constant, drop those that change. 5. OR the terms together. 6. Verify against the original truth table.

Numerical: What Minimisation Buys You in Silicon

Why minimisation still matters — one function, two very different costs

BEFORE — canonical SOP

- 7 AND gates (4-input)
- 1 OR gate (7-input)
- 4 inverters
- **28 literals**
- **≈ 12 gate-equivalents**
- **wide fan-in → slow**

AFTER — K-map minimised

- 2 AND gates (2-input)
- 1 AND gate (3-input)
- 1 OR gate (3-input)
- **6 literals**
- **≈ 5 gate-equivalents**
- **narrow fan-in → fast**

What you actually save

Area
≈ 58 % fewer gate-equivalents

Power
fewer switching nodes → less dynamic power

Delay
narrow fan-in gates switch faster

Yield
smaller die → more good dice per wafer

In practice a synthesis tool (Yosys, Design Compiler) runs Espresso/ABC-class algorithms for you — but you must be able to read the result and know when it is wrong.

Work it through with real numbers

Assume a 28 nm library where 1 gate-equivalent (GE) ≈ 0.5 µm² and a 2-input gate switching at 500 MHz burns ≈ 0.6 µW.

Before: 12 GE → 6.0 µm², ≈ 7.2 µW. **After:** 5 GE → 2.5 µm², ≈ 3.0 µW. **Saving: 58 % area and power.**

Now multiply by 200 000 instances of this block on a real SoC: 0.7 mm² of die and 0.84 W of dynamic power. That is the difference between a product that ships and one that does not.

Numbers are illustrative, chosen to show the method — always use your own library's datasheet.

TOPIC 3A · SCALING UP

Beyond K-Maps — What Real Tools Do

K-maps stop working past about 5 variables, and real blocks have hundreds of inputs. The algorithms below do the same job systematically. You will not implement them, but you must know their names and limits because they appear in every synthesis log.

Method	How it works	Scales to	Where you meet it
Karnaugh map	visual grouping on a Gray-coded grid	≤ 5–6 vars	exams, whiteboards, intuition
Quine–McCluskey	tabular; finds ALL prime implicants, then solves a covering problem	≤ ~15 vars	the exact algorithm K-maps approximate
Espresso	heuristic expand / reduce / irredundant loop — near-optimal, very fast	hundreds	classic two-level logic minimiser
ABC / AIG rewriting	multi-level: And-Inverter Graphs, cut rewriting, SAT sweeping	millions	inside Yosys and every commercial tool
BDD	canonical decision-diagram form of the function	varies wildly	equivalence checking, formal verification

So why learn K-maps at all?

Because you must be able to read what the tool produced and decide whether it is right. A synthesis report showing 4 000 gates for a function you know needs 40 means your RTL is wrong — an unintended latch, a runaway loop, or a missing don't-care. Minimisation intuition is your only defence against silently accepting bad hardware.

SUBTOPIC 3B

Combinational Logic Design

Circuits with no memory — the output is a pure function of the inputs, right now.

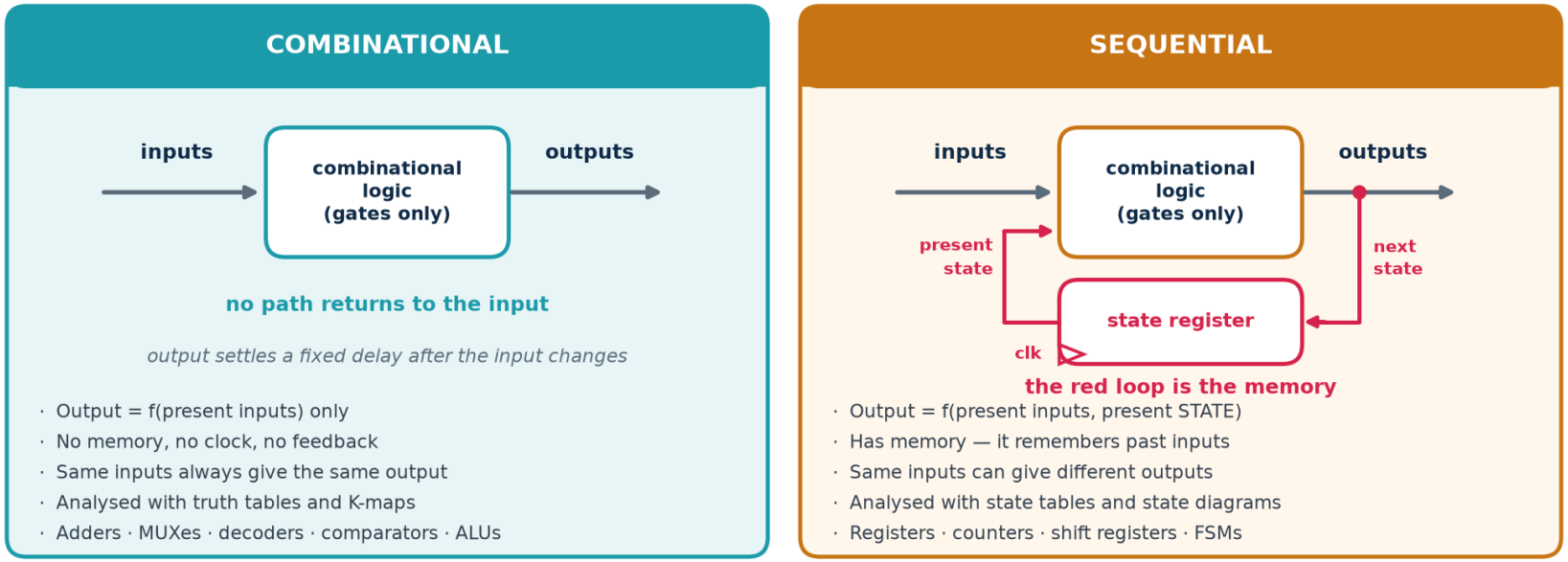
- The defining property, and the seven-step design procedure
- The standard building blocks: adders, MUX, decoders, comparators, ALUs
- What limits them: propagation delay, fan-in, fan-out and hazards
- How they are written in Verilog — and the inferred-latch trap

TOPIC 3B · DEFINITION

Combinational vs Sequential — The Only Real Difference

One structural feature separates the two families: a feedback path through a storage element. Everything else — that sequential circuits have memory, need a clock, and are analysed with state diagrams — follows from that one fact.

The two families of digital logic — the only structural difference is the feedback path



The formal definitions, worth memorising verbatim

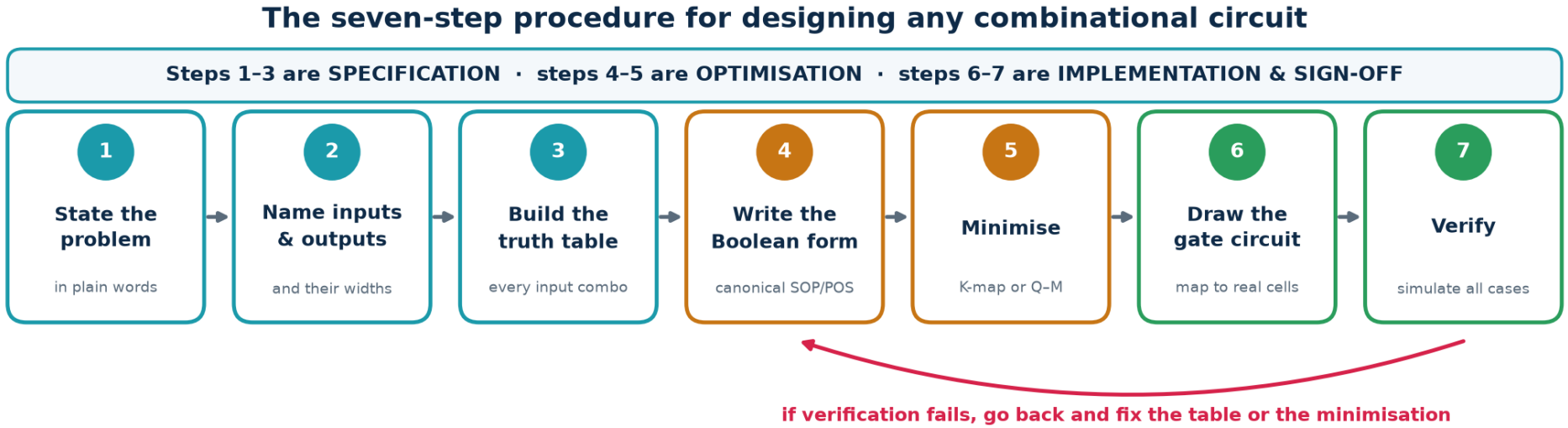
Combinational: the output at any instant depends ONLY on the inputs at that instant (after a bounded settling delay).

Sequential: the output depends on the present inputs AND on the sequence of past inputs, summarised in a finite amount of stored STATE.

TOPIC 3B · PROCEDURE

The Seven-Step Combinational Design Procedure

Follow this every time, even when the answer looks obvious. Most combinational bugs come from an incomplete truth table at step 3 — a missing row, or a case nobody thought about.



In an RTL flow you write steps 1–2 as a Verilog module header and step 3–6 as one always/assign block — the synthesiser performs steps 4–6 for you. Step 7 never goes away.

Worked micro-example — steps 1 to 3

1. 'Light a warning LED when at least two of three sensors read high.'
2. Inputs A, B, C (1 bit each); output W (1 bit).
3. Truth table: W = 1 for ABC = 011, 101, 110, 111 — four of the eight rows.

Steps 4 to 7

4. W = A'BC + AB'C + ABC' + ABC (canonical SOP)
5. K-map → W = AB + BC + AC — the majority function again.
6. Three 2-input ANDs into one 3-input OR. 7. Simulate all 8 input cases.

TOPIC 3B · TIERED DEPTH

Understanding Combinational Logic at Four Levels

The same idea at four depths — pick the one that fits the room.

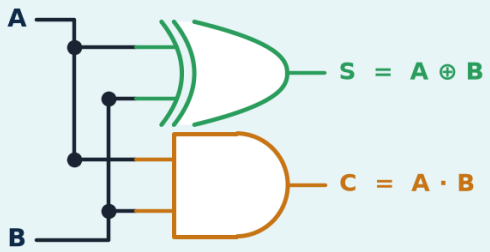
BASIC	A circuit with no memory. Change the inputs and, after a short delay, the outputs follow. Ask it the same question twice and you always get the same answer.
INTERMEDIATE	Built entirely from gates in an acyclic network. Fully described by a truth table. Standard blocks — adders, multiplexers, decoders, comparators — are reusable, well-understood patterns.
ADVANCED	Its speed is set by the critical path: the slowest input-to-output route, measured as a sum of gate delays plus wire RC. Multiple unequal paths to one output create hazards (glitches) even when the logic is correct.
INDUSTRY	Written as `assign` or `always @(*)` in RTL and left to synthesis, which restructures, resizes, buffers and technology-maps it against a timing constraint. Arithmetic is usually mapped to hand-tuned carry-lookahead or Wallace-tree macros, not to your literal gates.

Half Adder and Full Adder

Binary addition is the most-used combinational function on any chip. It is built from one tiny cell — the full adder — repeated once per bit. Derive it once and you understand every adder, subtractor, multiplier and ALU that follows.

Half adder and full adder — the atoms of every arithmetic unit

HALF ADDER — adds two bits



A	B	S	C
0	0	**0**	**0**
0	1	**1**	**0**
1	0	**1**	**0**
1	1	**0**	**1**

Limitation

It has no carry INPUT, so it can only be used for the least-significant bit of an adder.

FULL ADDER — adds three bits

$S = A \oplus B \oplus C_{in}$ $C_{out} = AB + C_{in}(A \oplus B)$

A	B	C_{in}	S	C_{out}
0	0	0	**0**	**0**
0	0	1	**1**	**0**
0	1	0	**1**	**0**
0	1	1	**0**	**1**
1	0	0	**1**	**0**
1	0	1	**0**	**1**
1	1	0	**0**	**1**
1	1	1	**1**	**1**

Built from

2 half adders + 1 OR gate

this is the cell that tiles into an n-bit adder

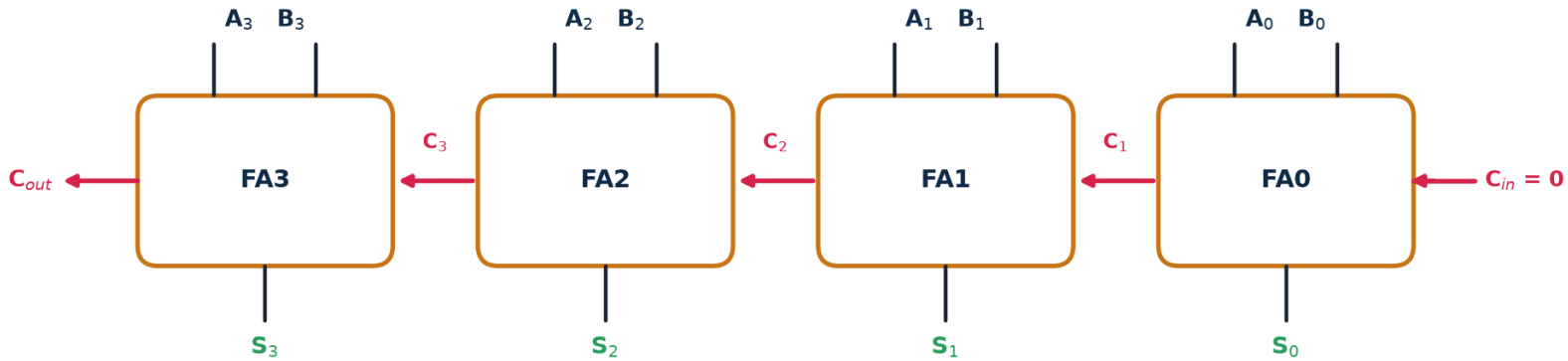
Subtraction comes free

A − B = A + (−B) = A + B' + 1. Feed B through XOR gates controlled by a *sub* signal (XOR with 1 inverts, XOR with 0 passes through) and tie that same *sub* to C_{in}. One adder now does both operations — this is exactly the ADDER / SUBTRACTOR block in the ALU slide.

4-Bit Ripple-Carry Adder — Correct but Slow

Chain n full adders and you have an n-bit adder. It is minimal in area and trivially correct — and its delay grows linearly with n, which makes it the classic critical path in any first-attempt design.

4-bit ripple-carry adder — correct, compact, and slow by construction



The problem: the carry must RIPPLE

FA3 cannot produce a correct sum until C_3 is valid, which needs C_2, which needs C_1…
Worst-case delay grows LINEARLY with the number of bits: $t_{RCA} \approx n \times t_{carry}$

The fix: carry-lookahead

Compute $G_i = A_iB_i$, $P_i = A_i \oplus B_i$ for all bits at once → delay grows as log n, at the cost of area.

Generate and Propagate

$G_i = A_iB_i$ — this bit generates a carry regardless of C_{in}.

$P_i = A_i \oplus B_i$ — this bit propagates an incoming carry.

$C_{i+1} = G_i + P_iC_i$

Carry-lookahead

Expand the recursion so every carry is a function of G/P only:

$C_2 = G_1 + P_1G_0 + P_1P_0C_0$ …

Delay ∝ log n instead of n — at the cost of much more area.

Others you will meet

Carry-select, carry-skip, Brent–Kung, Kogge–Stone prefix adders.

Synthesis picks one for you from a `+` in Verilog, guided by your timing constraint.

Numerical: Critical Path and f_{max} of a Ripple Adder

Given

A 4-bit ripple-carry adder built from full adders in a library where: carry-in to carry-out delay t_c = 90 ps, carry-in to sum delay t_s = 120 ps, and the input operands are driven by registers with t_cq = 60 ps into registers needing t_setup = 50 ps.

Answer the question in five steps — always state the path first

```
Step 1  Identify the critical path.
        It runs A0/B0 -> C1 -> C2 -> C3 -> S3  (the last SUM, not the last carry).

Step 2  Count the stages on that path.
        3 carry hops (FA0, FA1, FA2)  +  1 final sum (FA3)
        t_adder = 3 x t_c + t_s = 3 x 90 + 120 = 270 + 120 = 390 ps

Step 3  Add the register overhead to get the clock period.
        T_clk >= t_cq + t_adder + t_setup
               = 60 + 390 + 50 = 500 ps

Step 4  f_max = 1 / T_clk = 1 / 500 ps = 2.0 GHz

Step 5  Generalise to n bits:  t_adder = (n-1) x t_c + t_s
        n = 32  ->  31 x 90 + 120 = 2910 ps  ->  T_clk = 3020 ps  ->  f_max = 331 MHz
```

The lesson

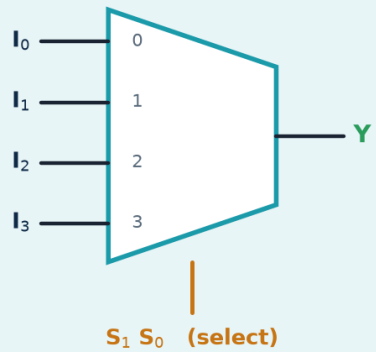
The 4-bit adder runs at 2 GHz; the same design at 32 bits collapses to 331 MHz. **Delay is linear in n, so frequency is roughly inverse-linear.** This is precisely the calculation that forces a designer to move from ripple-carry to carry-lookahead, or to pipeline the adder across two clock cycles.

Multiplexers and De-multiplexers

A multiplexer is a controllable switch: n select lines choose one of 2^n data inputs to appear at the output. It is the single most common structure in real hardware — every `if`, every `case`, every shared bus and every ALU function select becomes a MUX after synthesis.

Multiplexer and de-multiplexer — the data-routing pair

4-to-1 MULTIPLEXER — many in, one out

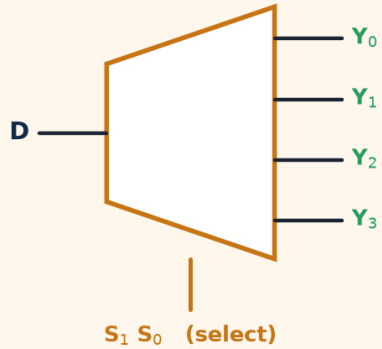


S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

n select lines choose one of 2^n inputs

1-to-4 DE-MULTIPLEXER — one in, many out



The selected output receives D; every other output is driven to 0.

A decoder is simply a de-multiplexer with D tied to 1

In Verilog, all of these are MUXes

```
assign y = sel ? a : b;
if (sel) y = a; else y = b;
case (sel) 2'b00: y = i0; ...
```

Why the size matters

A 2^n:1 MUX on an m-bit bus costs roughly $2^n \times m$ AND-OR pairs. A careless 16-way case on a 32-bit bus is 512 gates — and its delay grows with the number of select levels.

TOPIC 3B · A USEFUL TRICK

The Multiplexer as a Universal Logic Element

A 2^n:1 MUX can implement ANY function of n variables — just wire the truth-table output column to the data inputs and the variables to the select lines. This is not a curiosity: it is exactly how an FPGA look-up table (LUT) works.

The construction

Target: $F(A,B,C) = \Sigma m(1,3,5,6)$.

Use an 8:1 MUX with $S_2S_1S_0 = A,B,C$.

Tie $I_1 = I_3 = I_5 = I_6 = 1$ and every other data input to 0.

Done — no minimisation needed at all.

Halving the MUX

A 4:1 MUX also suffices: use A,B as select and feed each data input with one of 0, 1, C or C'.

Read pairs of truth-table rows: 00 → C, 01 → C, 10 → C, 11 → C'.

In general an n-variable function needs only a 2^{n-1}:1 MUX.

Why FPGAs are built this way

An FPGA logic cell is a small SRAM (typically a 6-input LUT = a 64:1 MUX with programmable data bits) plus a flip-flop.

Programming an FPGA literally means writing truth tables into those SRAM cells.

Consequence for your RTL

On an FPGA, a 6-input function and a 2-input function cost the SAME single LUT. Deliberately splitting logic into tiny pieces therefore buys you nothing and can cost you levels of delay. On an ASIC the opposite is true. **Know your target before you optimise.**

Decoders, Encoders and Priority Encoders

A decoder turns a compact binary code into one-hot select lines; an encoder does the reverse. Address decode, instruction decode, chip-select generation and interrupt arbitration are all this one pattern.

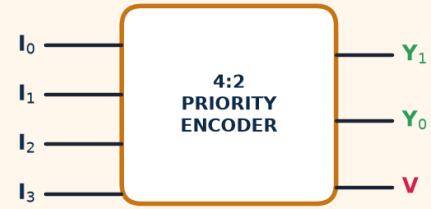
Decoders and encoders — converting between binary codes and one-hot lines

2-to-4 DECODER · binary code → one-hot

A_1	A_0	$D_3\ D_2\ D_1\ D_0$
0	0	0 0 0 1
0	1	0 0 1 0
1	0	0 1 0 0
1	1	1 0 0 0

Exactly ONE output is high at a time — this is how a CPU turns an address into a memory chip-select, and an opcode into a control line.

4-to-2 PRIORITY ENCODER · one-hot → binary code



$I_3\ I_2\ I_1\ I_0$	Y_1	Y_0	V
0 0 0 0	x	x	0
0 0 0 1	0	0	1
0 0 1 x	0	1	1
0 1 x x	1	0	1
1 x x x	1	1	1

'Priority' resolves several simultaneous 1s — the highest index wins. V (valid) distinguishes 'input 0 active' from 'no input active'.

Enable inputs

Real decoders have an EN pin: when EN = 0 every output is inactive. This is how several decoders are cascaded to build a larger one (two 2:4 + one 1:2 = a 3:8 decoder).

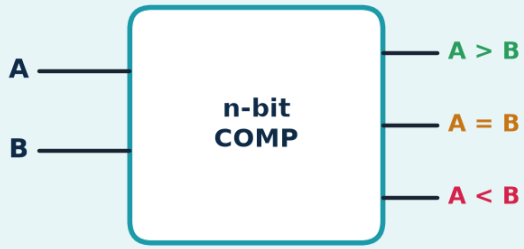
Why plain encoders are broken

A plain 4:2 encoder assumes exactly one input is high. If two are, it outputs the OR of their codes — a value that is simply wrong. A PRIORITY encoder resolves this, and the V (valid) output distinguishes 'input 0 active' from 'nothing active'.

Comparators, Parity and Other Standard Cells

Two more standard blocks you will meet in every datapath

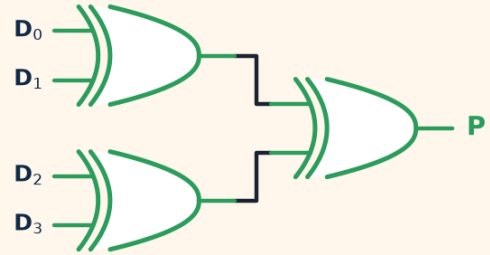
MAGNITUDE COMPARATOR



1-bit equality: $A = B \Leftrightarrow (A \oplus B)' = 1$

n-bit equality ANDs together n XNOR outputs. Magnitude compare works MSB-first: the first differing bit decides the result.

PARITY GENERATOR / CHECKER



$P = D_0 \oplus D_1 \oplus D_2 \oplus D_3$ (EVEN parity)

XOR of all bits = 1 when the number of 1s is odd. Send P alongside the data; the receiver re-computes it. A mismatch means at least one bit flipped in transit.

Comparator — building it up

`1-bit equal:  e = (A ⊕ B)'`

`n-bit equal:  AND all n of those`

Greater-than works MSB-first: A > B if the highest bit where they differ has A = 1. In Verilog this is just `A > B` and synthesis builds the ripple for you.

Parity — one XOR tree

Even parity bit P = XOR of all data bits.

Detects ANY odd number of bit flips, but cannot correct and misses even ones. For real protection use Hamming or ECC — the same XOR-tree idea, more bits.

Also in every library

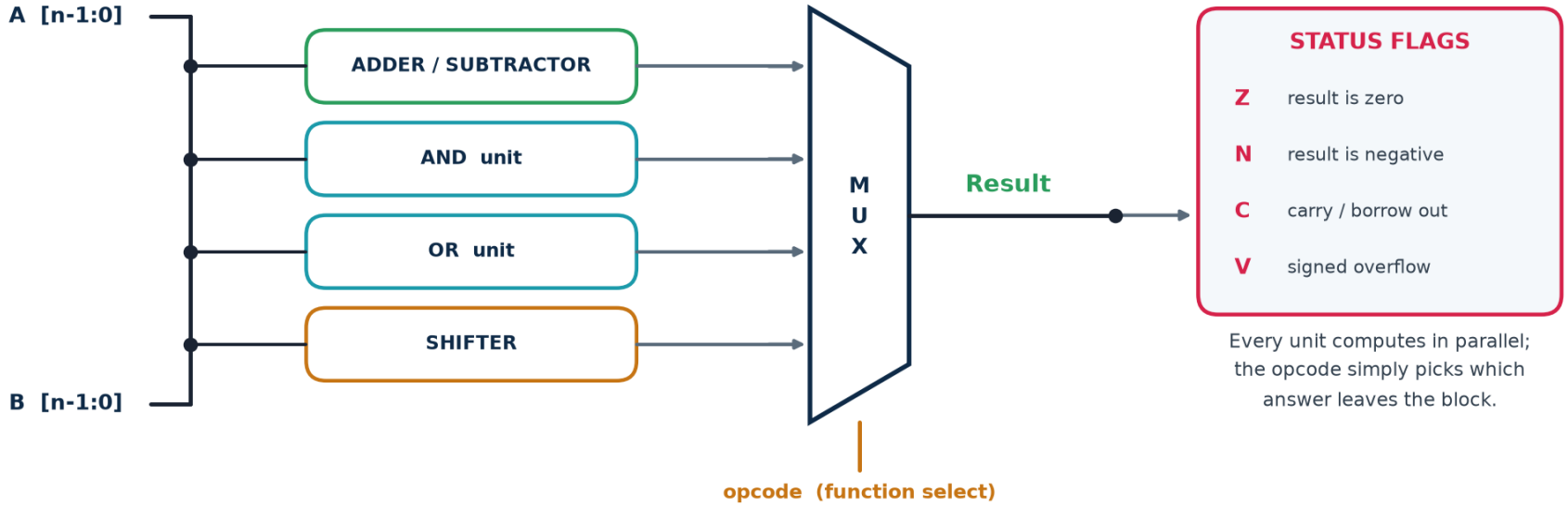
- Barrel shifter (a MUX per output bit)
- Priority arbiter
- Binary ↔ Gray converter
- Leading-zero counter

Trainer note: *ask the class to derive the 1-bit equality gate before you show it — XNOR-as-equality is the single most useful gate identity in this whole topic.*

An Arithmetic Logic Unit Is Just a MUX of Blocks

An ALU looks intimidating and is structurally trivial: run every operation in parallel on the same operands and use the opcode to select which answer leaves the block. It wastes power computing results you throw away — and it is fast, regular and easy to verify, which usually wins.

An ALU is just combinational blocks sharing one multiplexer



The status flags — and the mistake everyone makes with V

Z = result is all zeros (a NOR of every result bit). **N** = MSB of the result (the sign bit in two's complement). **C** = carry out of the MSB — meaningful for UNSIGNED overflow.

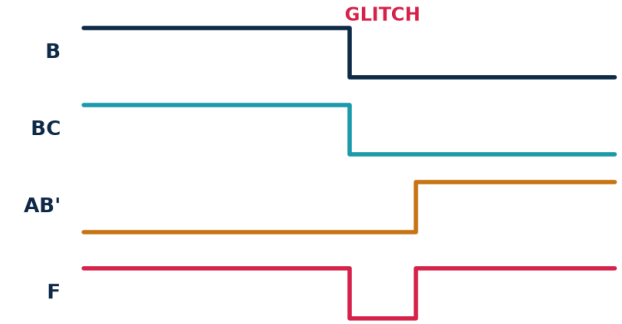
V = SIGNED overflow, and it is NOT the same as C. $V = C_n \oplus C_{n-1}$: the carry INTO the sign bit differs from the carry OUT of it. Adding two positives and getting a negative is the classic case.

Glitches — When Correct Logic Produces a Wrong Pulse

A hazard is a momentary wrong output caused purely by unequal path delays. The steady-state logic is perfectly correct; the transient is not. Understanding this is what makes the case for synchronous design.

Glitches and hazards — when correct logic still produces a wrong pulse

F = AB' + BC with A = C = 1 and B falling 1 → 0



BC falls one gate-delay BEFORE AB' rises — for that window neither term holds F high.

Glitches burn power, and they BREAK asynchronous logic — a glitch on a clock or reset line is fatal. This is a major reason industry designs synchronously.

Three kinds of hazard

Static-1	should stay 1, but dips to 0
Static-0	should stay 0, but spikes to 1
Dynamic	one transition that bounces 0→1→0→1

Root cause: two paths to the same output with unequal delay.

Two ways to deal with it

1. **Add a redundant (consensus) term**
   F = AB' + BC + AC. The extra AC group bridges the two K-map groups, so F is held high through the transition. It costs area and is logically redundant — but necessary.
2. **Or simply clock it**
   In a synchronous design the glitch settles long before the next clock edge, so no register ever samples it.

Where a glitch is harmless — and where it kills the chip

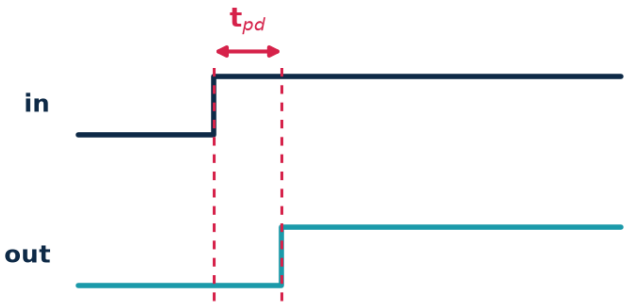
Harmless: feeding the D input of a flip-flop, provided it settles before the next clock edge. This is the normal case and why we design synchronously.

Fatal: on a clock line, a reset line, an asynchronous set/clear, a write-enable to memory, or any signal leaving the chip. Never generate a clock from combinational logic.

Propagation Delay, Fan-In and Fan-Out

What actually limits speed — propagation delay, fan-in and fan-out

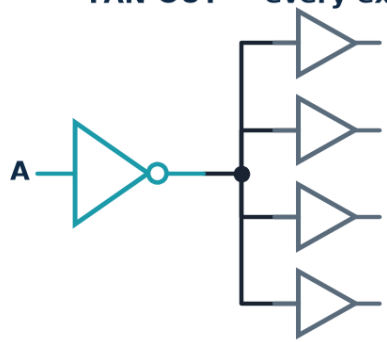
PROPAGATION DELAY of one gate



t_{pd} is measured at the 50 % points. It depends on the cell, the supply voltage, the temperature and — critically — the LOAD.

Path delay = the sum of t_{pd} along the slowest route

FAN-OUT — every extra load slows the driver



rule of thumb

$t_{pd} \approx t_{intrinsic} + k \cdot C_{load}$

double the loads, roughly double the load-dependent part

FAN-IN is the other half: a 6-input AND is slower than a tree of 2-input ANDs. Synthesis tools fix both automatically — by BUFFERING (adding repeaters) and by RESTRUCTURING logic.

Terms

t_pd propagation delay of one gate

Fan-in inputs on one gate

Fan-out loads driven by one output

Critical path slowest route through the block

Why fan-out costs time

Each load adds gate capacitance. Charging more capacitance through the same driver resistance takes longer — delay rises roughly linearly with load.

Fix: insert buffers, or size the driver up.

Why fan-in costs time

A CMOS gate stacks transistors in series per input, so a 6-input NAND is far slower than a tree of 2-input NANDs.

Fix: let synthesis restructure — or write balanced expressions yourself.

Static timing analysis (STA), covered in Topic 6, is nothing more than this arithmetic done automatically over every path in the design.

Writing Combinational Logic — and the Inferred-Latch Trap

Three ways to write pure combinational logic, and one way to get it catastrophically wrong.

Combinational always blocks: blocking assignments, complete assignment

```
// 1. Continuous assignment - always safe, always combinational
assign y = (a & b) | (~a & c);

// 2. always @(*) with BLOCKING assignments (=) and a DEFAULT
always @(*) begin
    y = 1'b0;                // <-- default first: kills every latch
    case (sel)
        2'b00: y = i0;
        2'b01: y = i1;
        default: y = 1'b0;
    endcase
end

// 3. WRONG - incomplete assignment infers a transparent LATCH
always @(*) begin
    if (enable) y = d;       // no else -> 'hold y' -> latch
end
```

The three rules

1. Use `=` (blocking) in combinational blocks, never `<=`.
2. Assign every output on every path — a default at the top is the easiest way.
3. Use `always @(*)` never a hand-written sensitivity list.

How to catch it

Yosys prints: `$_DLATCH_` cells in the statistics.

Commercial tools warn 'inferred latch for signal y'.

Treat every such warning as an ERROR. There is no case in this course where you want an unintended latch.

SUBTOPIC 3C

Sequential Logic, Flip-Flops, Registers and State Machines

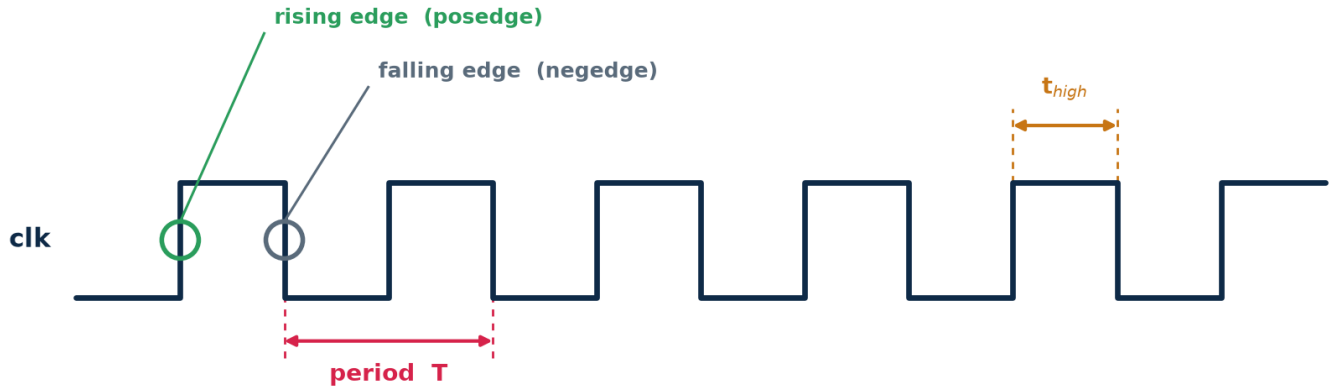
Circuits that remember — where time, the clock and state enter the design.

- The clock, latches, flip-flops and how edge-triggering is actually built
- Setup, hold, clock-to-Q, metastability, skew and the f_{max} calculation
- Registers, shift registers and counters
- Finite state machines: Moore, Mealy, encoding, and the Verilog template

The Clock — One Heartbeat for the Whole Design

Synchronous design means every storage element in a block changes at the same instant, on the same edge of one clock.
That single convention is what makes a billion-transistor chip analysable at all: it reduces 'does this circuit work?' to a set of arithmetic checks on one clock period.

The clock — the single heartbeat every synchronous design marches to



Period T	Frequency f = 1/T	Duty cycle	Edge
seconds per cycle	T = 2 ns → f = 500 MHz	t_{high} / T — usually 50 %	the instant a flip-flop samples

Worked conversions

T = 10 ns → f = 100 MHz. T = 2 ns → f = 500 MHz. T = 400 ps → f = 2.5 GHz.

f = 3.2 GHz → T = 312.5 ps.

Two rules that are never broken in RTL

1. Never generate a clock in combinational logic — use a clock enable instead.

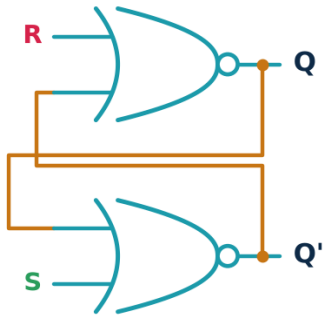
2. Never mix posedge and negedge of the same clock in one design without a very good, documented reason.

The SR Latch — Cross-Coupling Creates Memory

Take two NOR gates and feed each one's output back into the other. The circuit now has two stable configurations and will sit in whichever one you last pushed it into. That is memory — and it is the ancestor of every flip-flop, register and SRAM cell on the chip.

The SR latch — the smallest circuit that can remember one bit

Cross-coupled NOR gates



Each gate's output feeds the OTHER gate's input.
That loop is the memory: remove it and you have plain combinational logic.

Behaviour

S	R	Q_{next}	meaning
0	0	**Q**	HOLD — remembers
0	1	**0**	RESET to 0
1	0	**1**	SET to 1
1	1	**—**	FORBIDDEN

Why S = R = 1 is forbidden

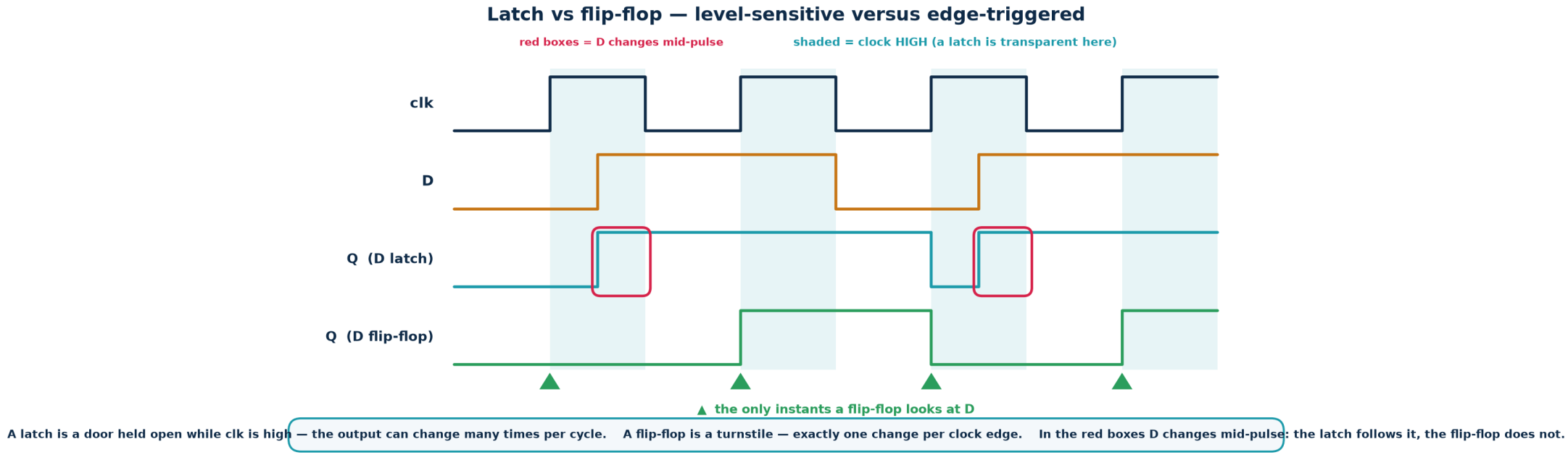
Both outputs are forced to 0, so Q and Q' are no longer complements.
Worse, releasing both together leaves the final state UNPREDICTABLE — a race.

Where you actually meet an SR latch

Rarely by choice in RTL — but it is the internal structure of every D latch, and a NAND version (active-low $\bar{S}\bar{R}$) is the standard hardware de-bouncer for a mechanical switch. It is also what an accidental combinational feedback loop in your Verilog turns into, which is why simulators warn about them.

Latch vs Flip-Flop — The Difference That Matters Most

A latch is LEVEL-sensitive: it is transparent for as long as its enable is high. A flip-flop is EDGE-triggered: it samples once, at one instant. Everything about timing analysis assumes flip-flops. Get this wrong and nothing downstream works.



Latch

Transparent while EN = 1

Output may change many times per cycle

Smaller and lower power

Hard to time; usually unintended

Flip-flop

Samples only on the clock edge

Exactly one change per cycle

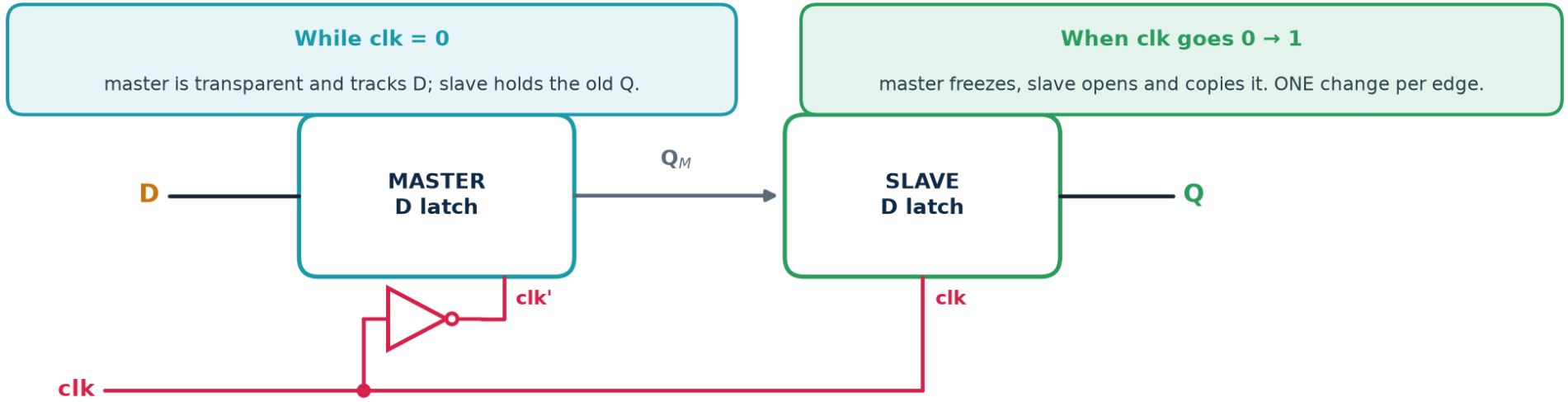
About twice the area of a latch

The only element you should infer

How Edge-Triggering Is Built — The Master–Slave Pair

A flip-flop is two latches in series driven by opposite clock phases. Because they are never transparent at the same time, data cannot race through both in one clock phase — and that, precisely, is edge-triggering.

How edge-triggering is actually built — the master-slave D flip-flop



Because the two latches are never transparent at the same time, data can never race through both in one clock phase — that is precisely what makes the flip-flop edge-triggered.

Why this matters to you as an RTL designer

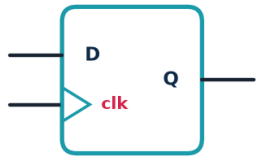
It explains where t_setup, t_hold and t_cq physically come from: setup is the time the master latch needs to capture, hold is the time before the master closes, and t_cq is the delay through the slave. It also explains why a flip-flop is roughly twice the area of a latch — you are paying for two storage nodes.

D, T, JK and SR Flip-Flops

Four behaviours, one clocked structure. In modern design only the D flip-flop is ever instantiated — but you must be able to read and convert between all four, because textbooks, exams and legacy schematics use them all.

The flip-flop family — four behaviours, one clocked structure

D flip-flop

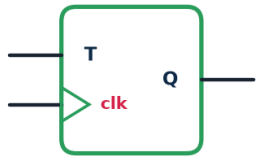


$Q_{next} = D$

D	Q_{next}
0	**0**
1	**1**

The workhorse. Everything synthesisable maps to this.

T flip-flop

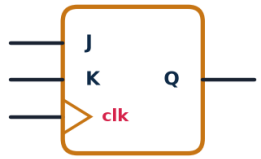


$Q_{next} = T \oplus Q$

T	Q_{next}
0	**Q**
1	**Q'**

Toggle on every edge when T=1 — the basis of counters.

JK flip-flop

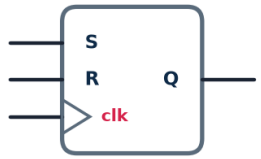


$Q_{next} = JQ' + K'Q$

J	K	Q_{next}
0	0	**Q**
0	1	**0**
1	0	**1**
1	1	**Q'**

Like SR but J=K=1 toggles instead of being illegal.

SR flip-flop



$Q_{next} = S + R'Q$

S	R	Q_{next}
0	0	**Q**
0	1	**0**
1	0	**1**
1	1	**—**

Clocked SR latch. S=R=1 remains forbidden.

Why the D flip-flop won

It has no forbidden state, no toggle surprise, and its next state is simply its input — which means synthesis can map any state equation directly onto it. Standard-cell libraries therefore contain dozens of D flip-flop variants (with/without reset, set, enable, scan) and no JK at all. **If you need a T or JK, build it from a D plus a little logic.**

TOPIC 3C · TABLES YOU NEED

Characteristic and Excitation Tables, and FF Conversion

A characteristic table answers 'given the inputs, what is the next state?'. An excitation table answers the reverse — 'to make this transition happen, what inputs do I need?'. You use the characteristic table to ANALYSE a circuit and the excitation table to DESIGN one.

Q → Q_next	D	T	J K	S R
0 → 0	0	0	0 ×	0 ×
0 → 1	1	1	1 ×	1 0
1 → 0	0	1	× 1	0 1
1 → 1	1	0	× 0	× 0

Characteristic equations

D-FF: $Q^+ = D$

T-FF: $Q^+ = T \oplus Q$

JK-FF: $Q^+ = J \cdot Q' + K' \cdot Q$

SR-FF: $Q^+ = S + R' \cdot Q \quad (SR \neq 11)$

Converting one to another

Put the target's excitation table beside the source's, K-map the required source inputs as functions of (present state, target inputs), and add that logic in front.

D from T: $D = T \oplus Q$ **T from D:** $T = D \oplus Q$

The × entries are don't-cares — and they are exactly what makes JK-based designs minimise so well on paper.

Reset — Synchronous vs Asynchronous

At power-up every flip-flop holds an unknown value. Reset is what forces the design into a known starting state. Choosing the wrong style, or releasing reset carelessly, produces bugs that only appear on real silicon.

The two templates — memorise both exactly

```
// ASYNCHRONOUS reset - reset appears in the sensitivity list
always @(posedge clk or posedge rst)
    if (rst) q <= 1'b0;
    else     q <= d;

// SYNCHRONOUS reset - reset is just another input to the D logic
always @(posedge clk)
    if (rst) q <= 1'b0;
    else     q <= d;
```

Asynchronous

Takes effect immediately, clock or no clock.

Essential at power-up, when the PLL has not locked and there is no clock yet.

Danger: its RELEASE is asynchronous and can violate recovery/removal time — so it must be de-asserted synchronously.

Synchronous

Takes effect only on a clock edge.

Fully covered by normal static timing analysis; no special constraints; filters glitches on the reset line.

Danger: useless if the clock is not running.

What industry actually does

Asynchronous assert, synchronous de-assert — the 'reset synchroniser'.

Two flip-flops clocked by the destination clock, with their async reset tied to the raw reset. Best of both.

Also note: on most FPGAs a global reset costs routing resources and is often unnecessary because the bitstream initialises every flip-flop. On an ASIC it is mandatory.

TOPIC 3C · TIERED DEPTH

Understanding Sequential Logic at Four Levels

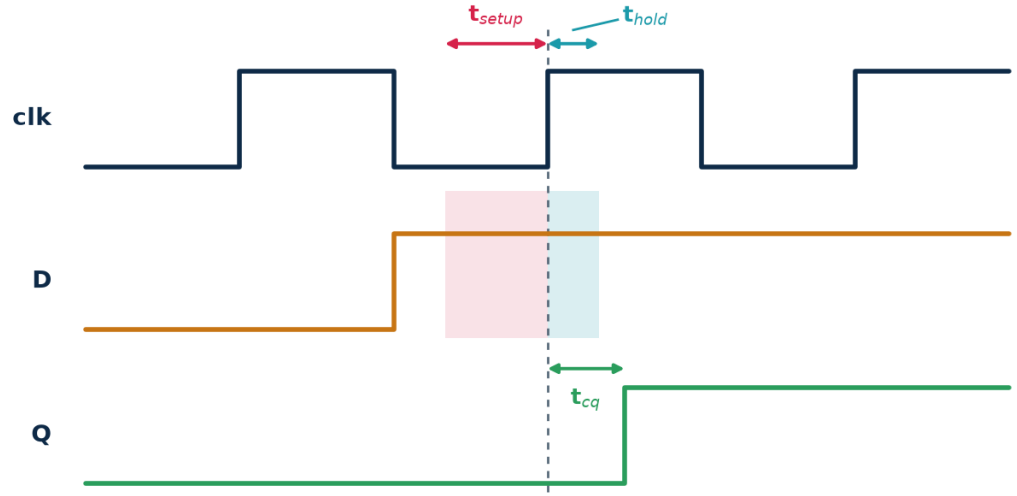
The same idea at four depths.

BASIC	A circuit that remembers. Its output depends not only on what you feed it now but on what happened before. A clock tells it when to update.
INTERMEDIATE	Combinational logic plus flip-flops in a feedback loop. The flip-flops hold the STATE; the logic computes the next state and the outputs. Counters, shift registers and controllers are all this shape.
ADVANCED	Correctness is a timing property, not just a logic one: every path must satisfy $T \geq t_{cq} + t_{logic} + t_{setup}$ and $t_{cq} + t_{logic,min} \geq t_{hold}$. Any input not synchronous to the clock must be synchronised or it will cause metastability.
INDUSTRY	Written as `always @(posedge clk)` with non-blocking assignments. Timing is closed by STA against SDC constraints; clock trees are balanced by CTS; every clock-domain crossing is reviewed as a first-class design artefact — CDC bugs are the leading cause of silicon respins.

Setup, Hold and Clock-to-Q

A flip-flop is not instantaneous. It requires the data to be steady for a window around the clock edge, and it takes time to produce its output afterwards. These three numbers come from the library datasheet and are the entire basis of timing analysis.

The three timing parameters that decide whether a flip-flop works



If you violate them

The flip-flop can go METASTABLE — its output hovers between 0 and 1 for an unbounded time, then settles to either value at random.

t_{setup}	D stable this long BEFORE the edge
t_{hold}	D stable this long AFTER the edge
t_{cq}	edge → Q valid (clock-to-Q delay)

Setup violation

Data arrives TOO LATE. Fix by shortening the logic path, or by slowing the clock. **A setup problem is a speed problem.**

Hold violation

Data arrives TOO EARLY and overwrites the value being captured. Fix by ADDING delay (buffers) on the short path. **Slowing the clock does not help at all.**

Clock-to-Q

Not a constraint but a cost — it eats into every clock period before your logic even starts. Faster (larger) flip-flops have smaller t_cq and larger area and power.

When the Rules Are Broken — Metastability

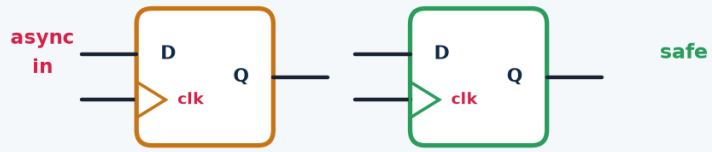
If data changes inside the setup/hold window, the flip-flop may enter a metastable state: its output hovers at an invalid intermediate voltage for an unbounded time, then resolves randomly to 0 or 1. It cannot be prevented — only made astronomically unlikely.

Metastability and the two-flop synchroniser

What a metastable output looks like

The resolution time is a random variable — it has NO upper bound.

The fix: a two-flop synchroniser



FF1 may go metastable; it is given a full clock period to settle before FF2 samples it. Two stages cut the failure rate to ~1 in 10^9 years.

Rule you must never break: every signal entering your clock domain from OUTSIDE it — another clock domain, a button, a sensor, an off-chip pin — must pass through a synchroniser first.

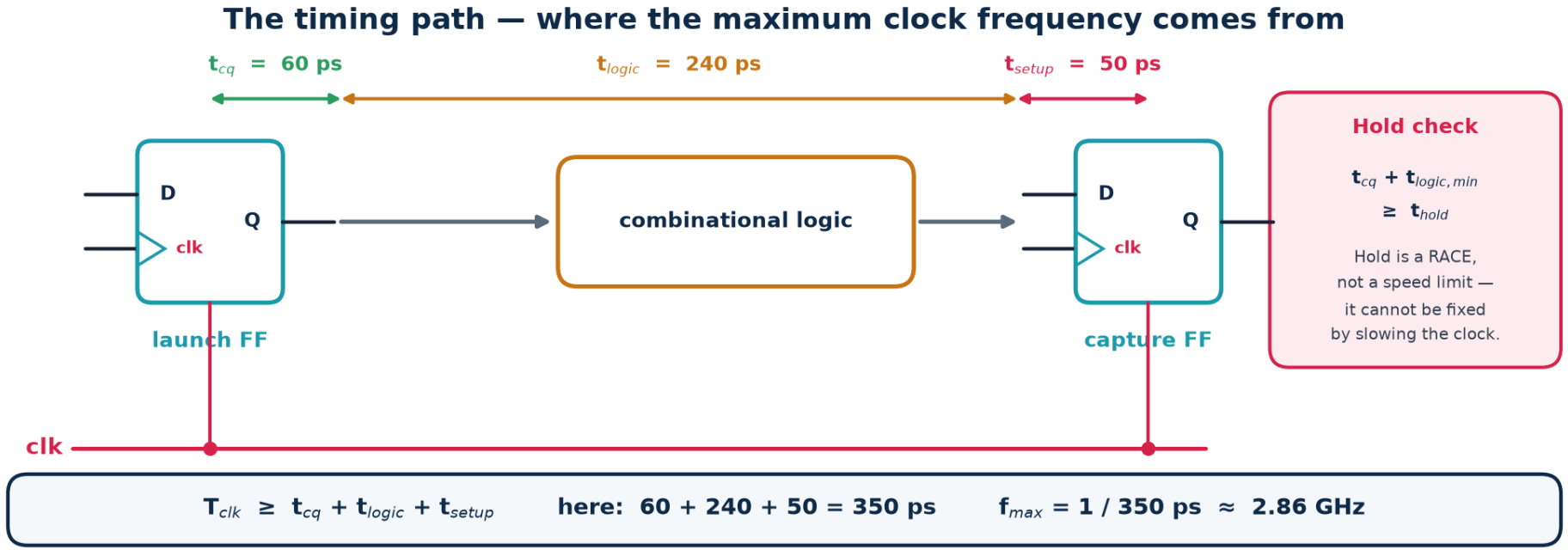
MTBF — the number you quote in a design review

$MTBF = e^{(t_r / \tau)} / (T_0 \cdot f_{clk} \cdot f_{data})$ where t_r is the resolution time you allow, τ and T_0 are library constants, and f_{clk}, f_{data} are the clock and data-change rates.

Because t_r sits in an exponential, adding ONE more synchroniser flip-flop typically moves MTBF from seconds to millions of years. That is why two stages is the universal minimum and three is used for very high-speed or safety-critical crossings.

Where the Maximum Clock Frequency Comes From

Every synchronous design reduces to this one picture: a launching flip-flop, some combinational logic, and a capturing flip-flop. The clock period must be long enough for a signal to get all the way across before the next edge arrives.



The two checks, and what each one means

Setup (max delay): T_clk ≥ t_cq + t_logic,max + t_setup + t_skew + t_jitter. The LONGEST path sets your maximum frequency.

Hold (min delay): t_cq + t_logic,min ≥ t_hold + t_skew. The SHORTEST path must still be long enough. This check is frequency-independent — a hold violation is broken at DC.

TOPIC 3C · PRACTICAL EXAMPLE 4

Numerical: f_{max}, Slack and the Hold Check

Given

t_cq = 60 ps, t_setup = 50 ps, t_hold = 40 ps, clock skew = 25 ps (capture clock arrives LATE), jitter = 15 ps. The combinational path between two flip-flops has t_logic,max = 240 ps and t_logic,min = 30 ps. Target frequency 2.0 GHz.

Do setup and hold as two separate calculations — never together

```
SETUP CHECK
  Required period  T_req = t_cq + t_logic,max + t_setup + t_jitter - t_skew
                         = 60 + 240 + 50 + 15 - 25          (late capture clock HELPS setup)
                         = 340 ps
  f_max = 1 / 340 ps = 2.94 GHz
  Target 2.0 GHz -> T_target = 500 ps
  SETUP SLACK = 500 - 340 = +160 ps          PASS  (positive slack = met)

HOLD CHECK
  Required:  t_cq + t_logic,min  >=  t_hold + t_skew
             60   + 30           >=  40     + 25
             90                  >=  65
  HOLD SLACK = 90 - 65 = +25 ps              PASS

WHAT IF the short path were only 5 ps of logic?
  60 + 5 = 65  >=  65   ->  slack = 0 ps, marginal; the tool would insert delay buffers.
```

Sign convention

Positive slack = requirement met, with margin to spare.

Negative slack = VIOLATION. The number tells you by how much.

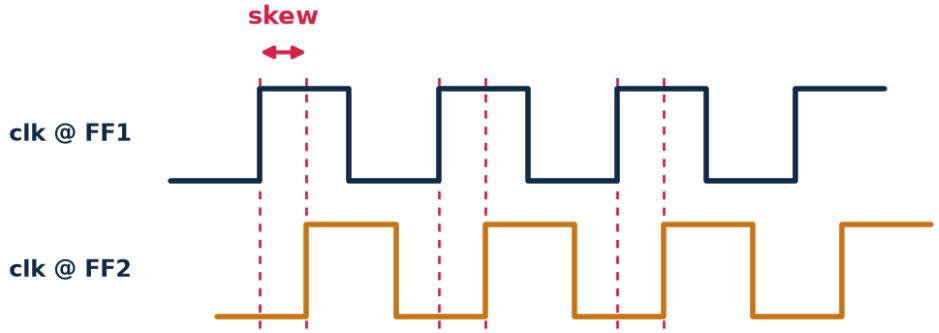
Note on skew

A late capture clock helps setup and hurts hold — which is why skew appears with opposite signs in the two checks. Getting that sign wrong is the single most common exam mistake.

Clock Skew and Jitter

The idealisation that 'the clock edge arrives everywhere at once' is false. Skew is a fixed difference between flip-flops; jitter is a random difference between cycles. Both are subtracted from the time available for your logic.

Clock skew and jitter — two ways the heartbeat goes wrong



SKEW — the same clock edge reaches different flip-flops at different times. Systematic, caused by unequal clock-tree wire lengths, and fixed by clock-tree synthesis (CTS).

JITTER

The edge of the SAME clock line arrives early or late from cycle to cycle — random, caused by supply noise and the PLL.

Both eat into your timing budget:

$$T_{clk} \geq t_{cq} + t_{logic} + t_{setup} + t_{skew} + t_{jitter}$$

Useful skew: a deliberately late capture clock buys the path extra time.

Skew — systematic

Cause: unequal clock-tree path lengths and loads.

Fixed by clock-tree synthesis (CTS), which inserts buffers to balance every branch to within a few tens of picoseconds.

Jitter — random

Cause: PLL noise, supply droop, crosstalk, temperature.

Cannot be designed out; budgeted as an uncertainty in the SDC constraints and paid for out of every clock period.

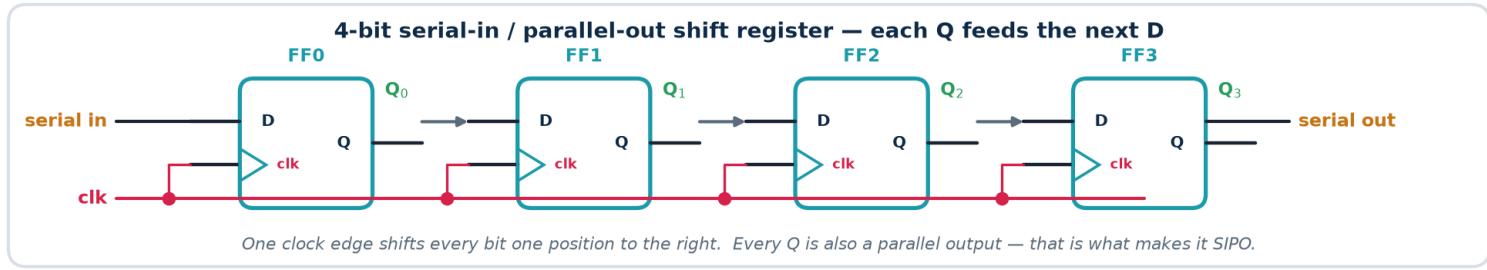
Useful skew

Deliberately delaying a capture clock borrows time for a slow path from the next stage. Powerful, and dangerous — it tightens the hold check on the same path.

Registers and Shift Registers

A register is simply n flip-flops sharing one clock. Wire each Q to the next D and you have a shift register — the basis of serial links, delay lines, scan chains for test, and multiply/divide by two.

Registers — flip-flops working as a group



SISO	SIPO	PISO	PIPO
serial in serial out	**serial in parallel out**	**parallel in serial out**	**parallel in parallel out**
1 bit in, 1 bit out, n cycles to fill	shift in bit by bit, read all n at once	load all n at once, shift out bit by bit	load and read in one edge — a plain register

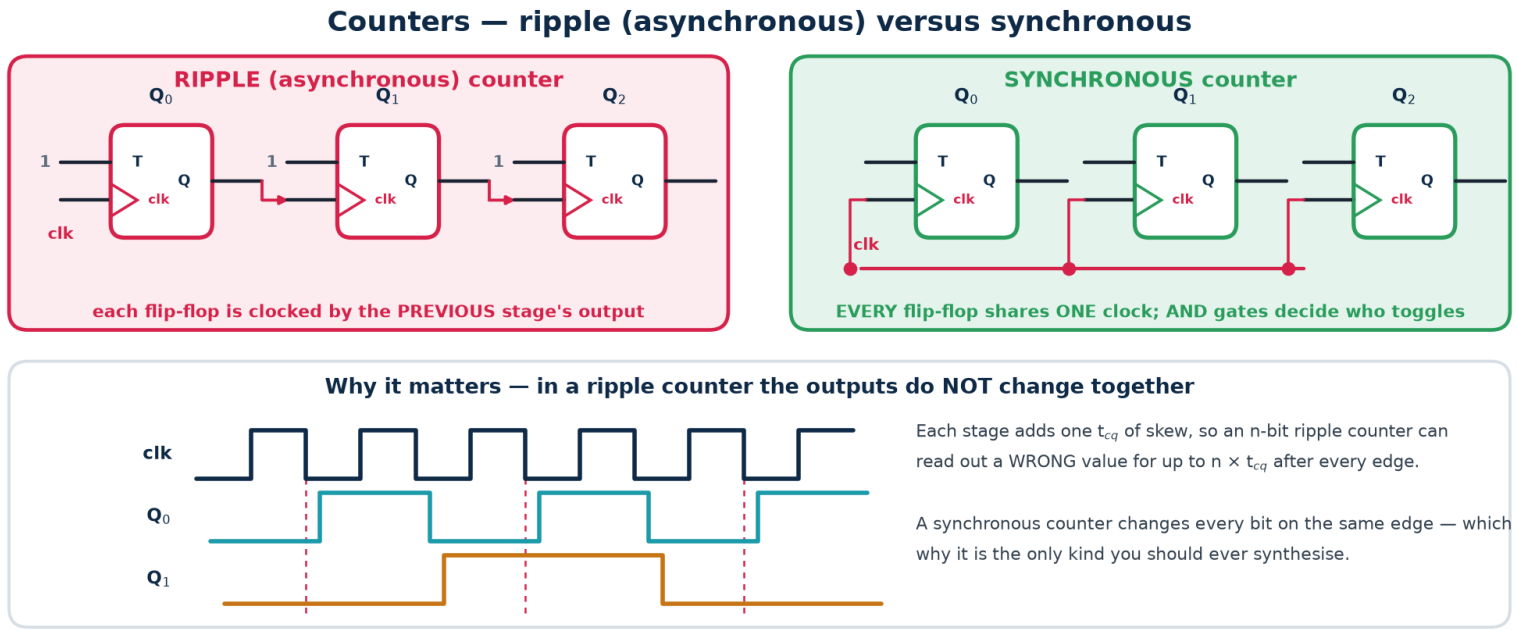
Applications you will actually build

Serialisation (SPI, UART, JTAG) · **Delay lines** and pipeline balancing · **Synchronisers** (a 2-bit shift register is exactly the 2-FF synchroniser) ·

Scan chains — during manufacturing test every flip-flop in the chip is rewired into one giant shift register so patterns can be loaded and results read out.
LFSRs — add an XOR feedback tap and you have a pseudo-random generator or a CRC engine.

Counters — Ripple vs Synchronous

A counter is a state machine whose state happens to be a number. There are two ways to build one, and only one of them belongs in a synthesised design.



Varieties you should know

Up / down / up-down · loadable · with enable · mod-N (divide by N) · ring counter (one-hot, N states from N flip-flops) · Johnson / twisted-ring (2N states from N flip-flops) · LFSR (2^n-1 states, maximal length, but not in numeric order).

Rule

Never write a ripple counter in RTL.

It creates a second clock domain out of a data signal, breaks static timing analysis, and produces transient wrong values. Always use one clock and an enable.

Worked: A Mod-10 (BCD) Synchronous Counter

Worked design — a mod-10 (BCD) synchronous up-counter

count	$Q_3Q_2Q_1Q_0$	next state	= count
0	0000	**0001**	1
1	0001	**0010**	2
2	0010	**0011**	3
3	0011	**0100**	4
4	0100	**0101**	5
5	0101	**0110**	6
6	0110	**0111**	7
7	0111	**1000**	8
8	1000	**1001**	9
9	1001	**0000**	0

the 1001 → 0000 wrap is the ONLY special row

Gate-level answer (D flip-flops)

$D_0 = Q_0'$

$D_1 = Q_1 \oplus (Q_0 \cdot Q_3')$

$D_2 = Q_2 \oplus (Q_1 \cdot Q_0)$

$D_3 = Q_3 \oplus (Q_3 \cdot Q_0 + Q_2 \cdot Q_1 \cdot Q_0)$

Derived from the excitation table, then K-map minimised.

…and how you would actually write it in RTL

```
always @(posedge clk or posedge rst)
  if (rst)            count <= 4'd0;
  else if (count == 4'd9) count <= 4'd0;
  else                count <= count + 1'b1;
```

Same circuit. The synthesiser derives those four equations for you — but you must be able to check that it did the right thing.

How the gate-level equations were obtained

1. Write the state table (0000 … 1001 and the wrap to 0000). 2. For each bit, tabulate $Q_i \rightarrow Q_i^+$. 3. Since $D = Q^+$, the D column IS the next-state column. 4. K-map each D_i over $Q_3Q_2Q_1Q_0$, treating states 1010–1111 as don't-cares. 5. Read off the minimal SOP.

Check one row by hand: at count 9 (Q = 1001), $D_0 = Q_0' = 0$, $D_1 = Q_1 \oplus (Q_0 \cdot Q_3') = 0 \oplus (1 \cdot 0) = 0$, $D_2 = Q_2 \oplus (Q_1 \cdot Q_0) = 0 \oplus 0 = 0$, $D_3 = Q_3 \oplus (Q_3Q_0 + Q_2Q_1Q_0) = 1 \oplus 1 = 0$. Next state = 0000. **Correct.**

Trap: states 1010–1111 are unreachable but not impossible — a glitch or SEU can land you there. Because we treated them as don't-cares the counter may lock up. A SAFE counter forces any illegal state back to 0000 explicitly.

Finite State Machines — The Universal Controller

An FSM is the formal name for 'a circuit that remembers where it is in a sequence'. Formally it is a 6-tuple (S, S_0, X, Y, δ, λ): a finite set of states, a start state, an input alphabet, an output alphabet, a next-state function δ, and an output function λ. Every protocol engine, bus controller, arbiter and CPU control unit is one.

The three pieces of hardware

1. State register — n flip-flops holding the present state.
2. Next-state logic — combinational, computes δ(state, input).
3. Output logic — combinational, computes λ.

Nothing else. Every FSM is this.

What counts as a 'state'

A state is an equivalence class of histories: two input histories are the same state if the machine will behave identically from now on.

This is why the 1011 detector needs five states — 'nothing', '1', '10', '101', '1011' — and not thirty-two.

Why finite matters

Finite states = finite flip-flops = a circuit you can actually build and exhaustively verify.

An FSM cannot count arbitrarily high or match arbitrarily deep nesting — for that you need a datapath (counter, stack) alongside it. That is the FSMD idea at the end of this section.

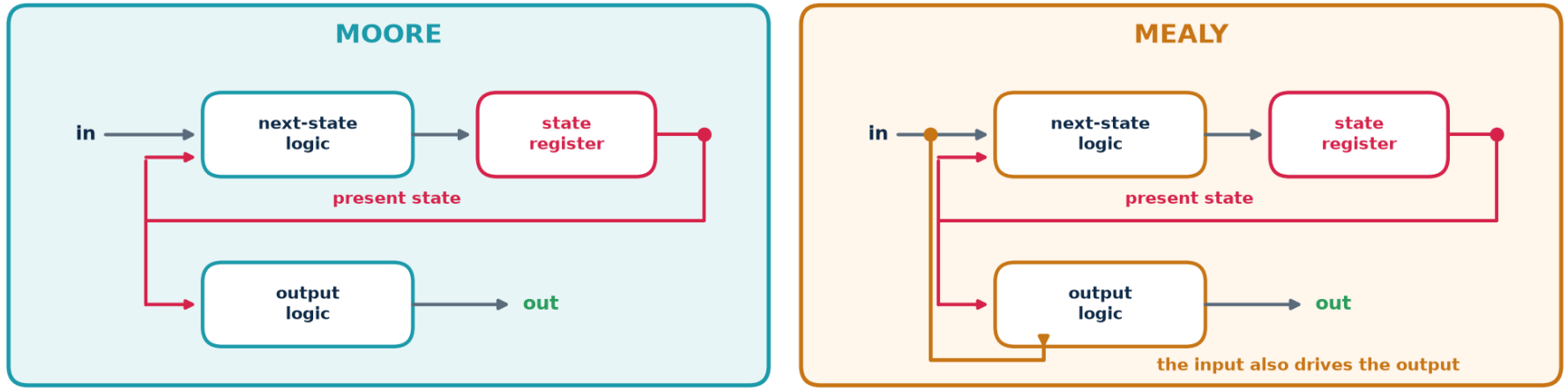
The Huffman model

Drawing the three blocks with the state register in a feedback loop is called the Huffman model of a sequential circuit. It is the same picture as the 'sequential' half of the very first diagram in subtopic 3b — every sequential circuit, however complicated, reduces to it.

Moore vs Mealy Machines

The only difference is whether the output logic can see the INPUT as well as the state. That one wire changes the timing behaviour, the state count and the glitch risk.

Moore vs Mealy — where the output logic gets its inputs



- Output = f(state) only
- Output changes only just after a clock edge
- Glitch-free, easy to time — safer
- Usually needs MORE states
- Reacts one cycle later

- Output = f(state, input)
- Output can change the moment the input does
- Can glitch — must be registered before use
- Usually needs FEWER states
- Reacts one cycle earlier

Which one should you use?

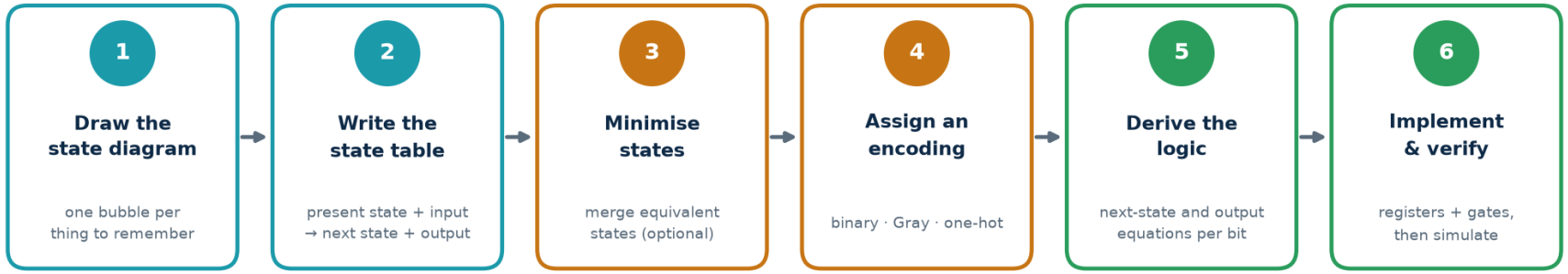
Default to Moore. Its outputs come straight off flip-flops, so they are glitch-free and easy to time. Use Mealy when you genuinely need the one-cycle-earlier response, and then **register the Mealy output** before it leaves the block — which, of course, turns it back into a Moore output one cycle later.

TOPIC 3C · PROCEDURE

The Six Steps of FSM Design

Do these in order, every time. The state diagram is the design; everything after it is mechanical. If the diagram is wrong, no amount of Verilog will save you.

The six steps of finite-state-machine design



In RTL you write steps 1–2 directly as a Verilog case statement and let the synthesiser do steps 3–5. Step 6 is still yours.

Two checks to run on the state diagram before you write any code

Completeness: from every state, is there exactly one outgoing arc for EVERY possible input value? A missing arc means an undefined transition — in hardware, a latch or a lock-up.

Determinism: is there at most one arc for each input value? Two arcs labelled '1' out of one state is not a circuit, it is a contradiction.

For a machine with s states and one binary input, count the arcs: there must be exactly 2s of them.

State minimisation (step 3) is optional in practice — synthesis tools do it, and modern FPGAs have flip-flops to spare. Understand it; do not agonise over it.

State Encoding — Binary, Gray and One-Hot

The states in your diagram are names; the flip-flops hold bits. How you map one to the other changes the flip-flop count, the amount of next-state logic, the speed and the power — without changing the behaviour at all.

State encoding — the same FSM, three different silicon costs

state	BINARY	GRAY	ONE-HOT
S_0	**000**	**000**	**00001**
S_1	**001**	**001**	**00010**
S_2	**010**	**011**	**00100**
S_3	**011**	**010**	**01000**
S_4	**100**	**110**	**10000**

BINARY	GRAY	ONE-HOT
$\lceil \log_2 n \rceil$ flip-flops — fewest	Same FF count as binary	n flip-flops — most
More next-state logic	One bit changes per step	Next-state logic is trivial
Default for large FSMs	Less switching → less power	Fastest; ideal on FPGAs
5 states → 3 FFs	Good for counters/pointers	5 states → 5 FFs

Verilog: use `localparam` (or an enum) for state names and let the tool pick the encoding — then CHECK the synthesis report.

How to choose

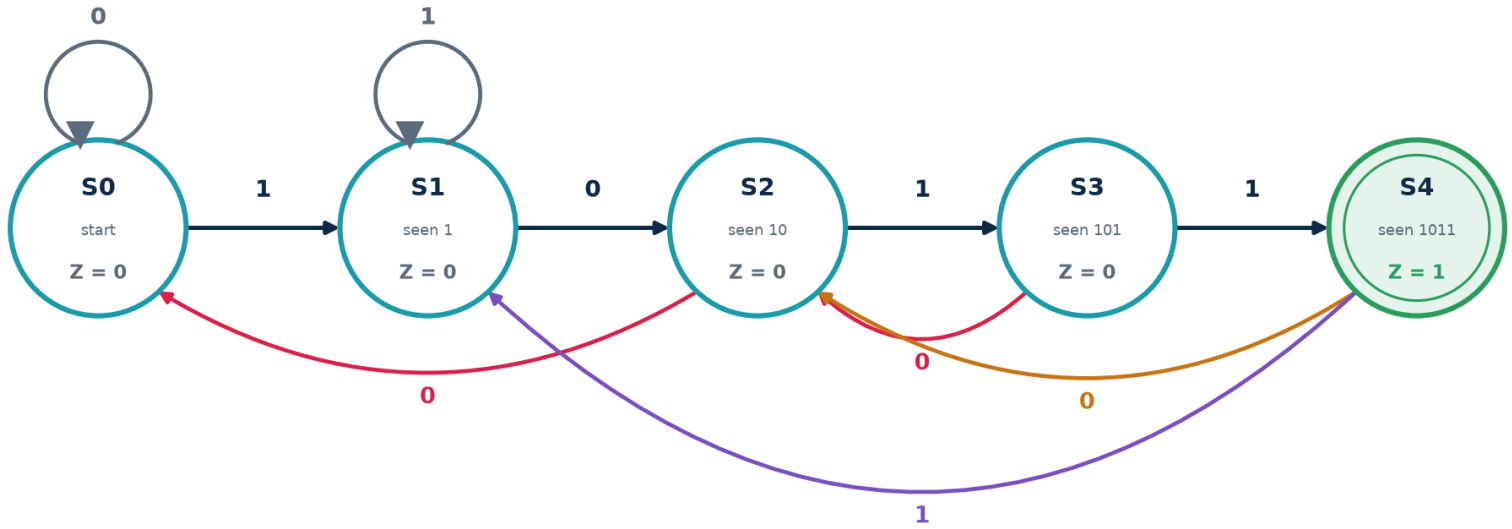
FPGA: one-hot, almost always. Flip-flops are free and abundant; LUT depth is what costs you speed, and one-hot next-state logic is one level deep.

ASIC with many states: binary, to keep the register count and hence area down. **Low-power or clock-crossing:** Gray, because only one bit toggles per transition — less switching energy and safe to sample across a clock domain.

Designing a '1011' Sequence Detector — Step 1

Specification. A serial input X arrives one bit per clock. Assert output Z whenever the last four bits received were 1, 0, 1, 1. Overlapping sequences count — so the stream 1011011 contains TWO occurrences.

Moore state diagram — a '1011' sequence detector (overlapping)



Each state remembers HOW MUCH of the pattern has matched so far — that is all a state ever is.

'Overlapping' means that after a hit we keep the longest useful suffix, so S4 on input '1' goes to S1, not back to S0.

How each state was chosen

Ask: 'what is the longest PREFIX of 1011 that is currently a suffix of what I have received?' That question has exactly five answers — none, 1, 10, 101, 1011 — so there are five states, and the answer to the question IS the state. **This suffix-matching insight generalises to any pattern detector.**

'1011' Detector — Steps 2 to 5: Table, Encoding, Logic

Present state	Encoding $Q_2Q_1Q_0$	Next state (X=0)	Next state (X=1)	Z
S0 (start)	000	S0 000	S1 001	**0**
S1 (1)	001	S2 010	S1 001	**0**
S2 (10)	010	S0 000	S3 011	**0**
S3 (101)	011	S2 010	S4 100	**0**
S4 (1011)	100	S2 010	S1 001	**1**

Derive, then ALWAYS re-check against the table row by row

```
Next-state equations   (K-map each D bit over Q2 Q1 Q0 X;  codes 101-111 are don't-cares)

  D2 = Q1 . Q0 . X                    only S3 (011) with X=1 reaches S4

  D1 = X' . (Q2 + Q0)  +  Q1 . Q0' . X
       ^^^^^^^^^^^^^^     ^^^^^^^^^^^^
       S1,S3,S4 on X=0    S2 on X=1  (010 -> 011)

  D0 = X . (Q1 . Q0)'                 every X=1 arc lands on S1 or S3, both odd,
                                      except S3 (011) which goes to S4 (100)

Output equation  (Moore - a function of the state alone)
  Z  = Q2                             S4 is the only state whose code has Q2 = 1
```

Check one row by hand: S3 = 011 with X = 1 gives $D_2 = 1{\cdot}1{\cdot}1 = 1$, $D_1 = 0{\cdot}(\ldots) + 1{\cdot}0{\cdot}1 = 0$, $D_0 = 1{\cdot}(1{\cdot}1)' = 0$ → next state 100 = S4. ✓ Notice too that $Z = Q_2$ fell out for free because we chose the encoding well. All of this is what synthesis does for you when you write the case statement on the next slide.

'1011' Detector — The Three-Block Verilog FSM

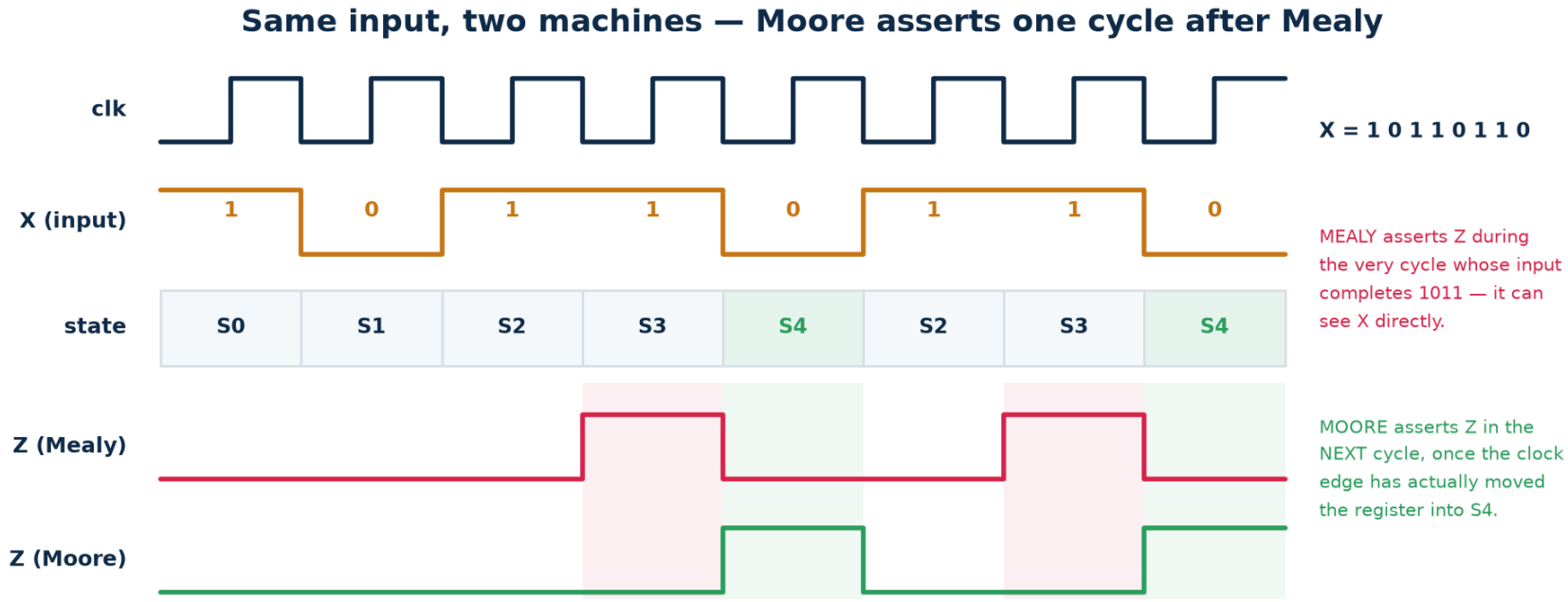
The three-always-block template — use this shape for EVERY FSM you write

```
module seq_detect_1011 (input clk, input rst_n, input x, output reg z);

    localparam S0=3'd0, S1=3'd1, S2=3'd2, S3=3'd3, S4=3'd4;
    reg [2:0] state, next;

    // ---- BLOCK 1 : state register  (sequential, non-blocking) ----
    always @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0;
        else        state <= next;

    // ---- BLOCK 2 : next-state logic  (combinational, blocking) ----
    always @(*) begin
        next = state;                       // default: no latch
        case (state)
            S0: next = x ? S1 : S0;
            S1: next = x ? S1 : S2;
            S2: next = x ? S3 : S0;
            S3: next = x ? S4 : S2;
            S4: next = x ? S1 : S2;
            default: next = S0;             // safe FSM: recover from illegal states
        endcase
    end

    // ---- BLOCK 3 : output logic  (Moore - state only) ----
    always @(*) z = (state == S4);

endmodule
```

Why three blocks and not one

It separates the clocked element from the combinational logic, so the synthesised hardware matches the Huffman model exactly. It makes the reset behaviour explicit, keeps each block short enough to read, and makes latch inference impossible if you keep the default assignment. **One-block FSMs are legal and much harder to debug.**

Trace: Moore and Mealy on the Same Input Stream



Read the trace with the class, cycle by cycle

Input X = 1 0 1 1 0 1 1 0. Both machines see the fourth bit complete the pattern.

Mealy asserts Z during cycle 4 — combinationally, the moment X goes high while the state is S3. **Moore** asserts Z during cycle 5 — the clock edge at the end of cycle 4 moved the register into S4, and only then does $Z = Q_2$ become 1.

Second detection: overlapping means the '11' at the end is reused, so the pattern completes again on bit 7 and both machines fire a second time.

Exam question you should be able to answer instantly: 'same behaviour, so why choose one?' — Mealy is one cycle faster and needs fewer states; Moore is glitch-free and trivially timeable.

TOPIC 3C · PITFALLS

Five Ways an FSM Goes Wrong in Silicon

Failure	What happens	How you prevent it
Unreachable / illegal states	With 5 states in 3 bits, codes 101–111 exist in hardware but not in your diagram. A glitch or radiation event lands you there.	Write a `default` branch that returns to a safe state — a SAFE FSM.
Deadlock / lock-up	A group of states with no path back to normal operation. The block hangs and only a reset recovers it.	Prove reachability: from every state, a path to the start state must exist.
Incomplete transitions	A state with no arc for some input value. In RTL this becomes an inferred latch.	Assign `next` a default value at the top of the always block.
Unregistered Mealy output	The glitchy combinational output drives a clock enable or a memory write. Random corruption.	Register the output, or use Moore.
Asynchronous input straight into the FSM	A button or a signal from another clock domain violates setup/hold and the FSM enters an illegal state.	Two-flop synchroniser on EVERY asynchronous input.

The one-line summary

A state machine is only as reliable as its handling of the cases you did NOT draw. Always write the default branch, always synchronise the inputs, always simulate an illegal-state injection.

Sequential Logic — Blocking vs Non-Blocking

This is the single most common Verilog error in the industry. Both versions below simulate; only one synthesises to what you meant, and simulation and hardware then disagree — the worst class of bug there is.

Same intent, different hardware

```
// CORRECT - non-blocking (<=) in a clocked block gives a true 2-stage shift register
always @(posedge clk) begin
    q1 <= d;
    q2 <= q1;          // q2 gets the OLD q1  ->  two flip-flops in series
end

// WRONG - blocking (=) makes q1 update immediately, so q2 gets the NEW q1
always @(posedge clk) begin
    q1 = d;
    q2 = q1;           // q2 gets d  ->  ONE flip-flop, and a race in simulation
end
```

The rule — no exceptions

Non-blocking `<=` in every `always @(posedge clk)` block.

Blocking `=` in every `always @(*)` block.

Never mix the two in one block. Never assign the same signal from two blocks.

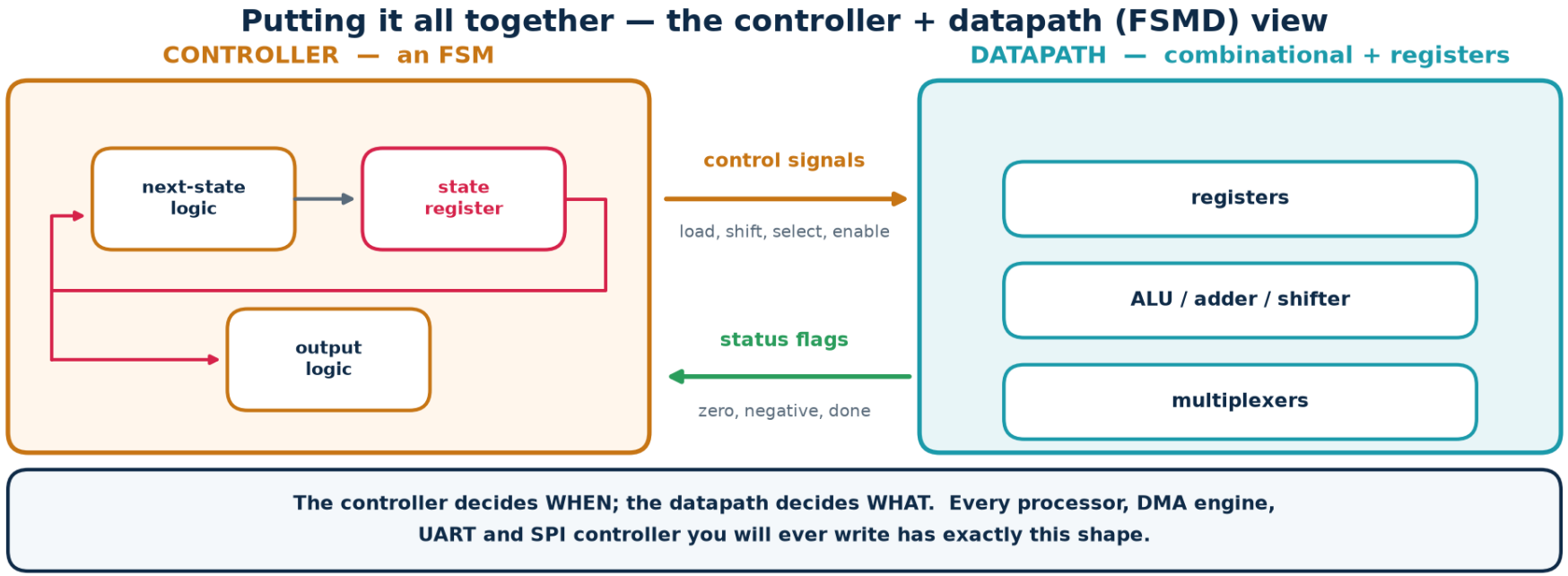
Why the rule exists

Non-blocking assignments all sample their right-hand sides first, then update together at the end of the time step — which is exactly what a bank of flip-flops does on a clock edge.

Blocking assignments execute in order, like software, which models a wire, not a register.

Controller Plus Datapath — How Real Blocks Are Built

Everything in this topic now assembles into one architecture. The combinational blocks from 3b form the datapath; the FSM from 3c forms the controller; control signals go one way and status flags come back. This partition is how every non-trivial digital block is organised.



A concrete example you can build in one afternoon

A serial multiplier. Datapath: a shift register for the multiplier, an adder, an accumulator register, a counter. Controller: an FSM with states IDLE → LOAD → ADD/SHIFT (looping) → DONE, using the counter's 'terminal count' status flag to decide when to leave the loop and driving the datapath's load, add and shift enables.

HANDS-ON

Tools, Installation and Tutorials

Everything from here is done at a keyboard. The tools are free and take about fifteen minutes to install.

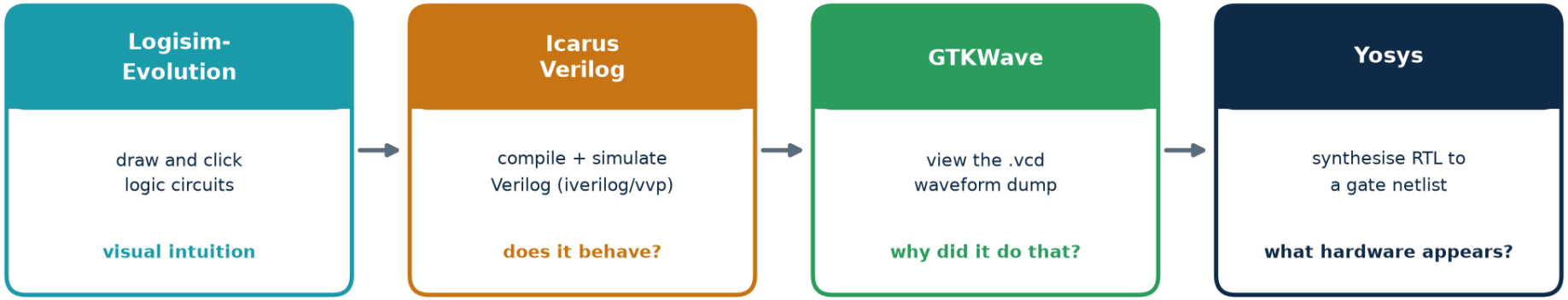
- Install: Logisim-Evolution, Icarus Verilog, GTKWave, Yosys
- T1 — Build and simulate gates, an adder and an FSM visually in Logisim
- T2 — Compile and simulate a full adder and a 4-bit adder; read the waveform
- T3 — Flip-flops, a shift register and a mod-10 counter
- T4 — The 1011 FSM: simulate it, then synthesise it and read the netlist

HANDS-ON · TOOLCHAIN

The Four Tools and What Each One Is For

The open-source toolchain used in this topic's labs

one flow: draw it → describe it in Verilog → simulate it → look at it → synthesise it



All four are free, open-source and run on Windows, Linux and macOS.

Ubuntu / WSL: sudo apt install iverilog gtkwave yosys macOS: brew install icarus-verilog gtkwave yosys Windows: use WSL2, or OSS CAD Suite

Why this particular set

They are free, open-source, small, cross-platform and script-driven — so every student can run the identical flow at home, and the whole lab can be checked automatically. The commercial equivalents (ModelSim/Questa, VCS, Design Compiler, Vivado) do the same jobs with the same concepts and a larger price tag.

Java note: Logisim-Evolution needs a Java Runtime (JRE 17 or later). Everything else is native.

HANDS-ON · INSTALLATION

Installing the Toolchain — Step by Step

Windows (recommended: WSL2)

1. Open PowerShell as Administrator:

```
wsl --install -d Ubuntu
```

2. Reboot, then open the Ubuntu terminal.
3. Run the Ubuntu commands opposite.
4. For GTKWave graphics, install VcXsrv or use WSLg (Win 11).

Alternative without WSL: download the OSS CAD Suite ZIP, unzip, and run environment.bat.

Ubuntu / Debian / WSL

```
sudo apt update
sudo apt install -y iverilog \
    gtkwave yosys default-jre
```

Logisim-Evolution: download the .jar from its GitHub releases page and run:

```
java -jar logisim-evolution.jar
```

macOS (Homebrew)

```
/bin/bash -c "$(curl -fsSL \
  https://raw.githubusercontent.com/\
  Homebrew/install/HEAD/install.sh)"

brew install icarus-verilog \
    gtkwave yosys temurin
```

Verification — run this before the lab, not during it

```
# ---- VERIFY the installation - every one of these must print a version ----
iverilog -V   | head -1        # expect: Icarus Verilog version 11.x or 12.x
vvp   -V      | head -1        # the Icarus runtime engine
gtkwave --version | head -1    # expect: GTKWave Analyzer v3.3.x
yosys -V                       # expect: Yosys 0.3x
java -version                  # expect: openjdk 17 or later
```

Troubleshooting: 'command not found' after apt succeeded → close and reopen the terminal. GTKWave opens but shows nothing → you forgot $dumpfile/$dumpvars in the testbench. Yosys 'read_verilog: syntax error' → Icarus accepts some non-standard code that Yosys rejects; fix the RTL, do not work around it.

HANDS-ON · TUTORIAL T1

Logisim-Evolution — See the Logic Before You Code It

Goal: build three circuits by clicking, poke the inputs, and watch the wires change colour. Thirty minutes here saves hours of confusion later.
Deliverable: three .circ files plus a screenshot of each truth table.

T1.1 — A half adder from scratch (10 min)

1. Launch: `java -jar logisim-evolution.jar`
2. From the toolbar place two Input pins, label them A and B (right-click → Edit label).
3. From the Gates library drag in one XOR and one AND gate.
4. Place two Output pins, label them S and C.
5. Wire A and B to both gates; XOR → S, AND → C.
6. Press the poke (hand) tool and click A and B to toggle them. Verify all four rows against the truth table on slide 23.

T1.2 — 4-bit ripple adder (15 min)

· Project → Add Circuit… name it full_adder; build it from two half adders + an OR.

· Back in main, place four copies of your full_adder from the project tree.

· Chain Cout → Cin. Drive A and B from 4-bit input pins.

· Add 5, add 3, check you get 8. Then add 15 + 1 and confirm Cout goes high.

T1.3 — The 1011 detector (20 min)

· Place a 3-bit Register and a Clock from the Memory / Wiring libraries.

· Implement the next-state equations from slide 54 with gates.

· Simulate → Tick Enabled, then hand-enter 1,0,1,1 on X and watch Z pulse.

· Keep going with 0,1,1 and confirm the OVERLAPPING second hit.

Checkpoint question for the class: *in T1.3, what happens if you tick the clock while X is changing? Relate the answer to the metastability slide.*

HANDS-ON · TUTORIAL T2

Icarus Verilog + GTKWave — Simulate a Full Adder

Goal: take the same circuit you clicked together in T1 and describe it in Verilog, then prove it correct with a self-checking testbench and look at the waveform.

Two files — the design and a self-checking testbench

```
// ---------- full_adder.v ----------
module full_adder (input a, input b, input cin, output sum, output cout);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule

// ---------- tb_full_adder.v ----------
`timescale 1ns/1ps
module tb_full_adder;
    reg a, b, cin;  wire sum, cout;  integer i;  integer errors = 0;
    full_adder dut (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

    initial begin
        $dumpfile("fa.vcd");  $dumpvars(0, tb_full_adder);   // <-- waveform dump
        for (i = 0; i < 8; i = i + 1) begin
            {a, b, cin} = i[2:0];  #10;
            if ({cout, sum} !== (a + b + cin)) begin
                $display("FAIL a=%b b=%b cin=%b -> %b%b", a, b, cin, cout, sum);
                errors = errors + 1;
            end
        end
        if (errors == 0) $display("PASS - all 8 cases correct");
        $finish;
    end
endmodule
```

Run it — three commands

```
iverilog -g2012 -o fa.out full_adder.v tb_full_adder.v   # compile
vvp fa.out                                               # run -> 'PASS - all 8 cases correct'
gtkwave fa.vcd &                                         # view the waveform
# In GTKWave: pick tb_full_adder in the SST pane, select all, Insert, then 'zoom fit'.
```

HANDS-ON · TUTORIAL T3

Sequential Logic — Flip-Flop, Shift Register, Counter

Goal: see with your own eyes that a flip-flop samples only on the edge, that non-blocking assignment builds a real shift register, and that a mod-10 counter wraps at nine.

Three designs — build them in one file or three

```
// ---------- dff.v : the reference D flip-flop, async active-low reset ----------
module dff (input clk, input rst_n, input d, output reg q);
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 1'b0;
        else        q <= d;
endmodule

// ---------- shift4.v : four-stage shift register ----------
module shift4 (input clk, input rst_n, input sin, output reg [3:0] q);
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 4'b0000;
        else        q <= {q[2:0], sin};      // shift left, sin enters at bit 0
endmodule

// ---------- bcd_counter.v : mod-10 synchronous up-counter ----------
module bcd_counter (input clk, input rst_n, input en, output reg [3:0] cnt);
    always @(posedge clk or negedge rst_n)
        if (!rst_n)        cnt <= 4'd0;
        else if (en) begin
            if (cnt == 4'd9) cnt <= 4'd0;
            else             cnt <= cnt + 4'd1;
        end
endmodule
```

What to look for in the waveform

· q changes ONLY at rising clk edges, never when d moves mid-cycle.

· The shift register's bit 3 lags sin by exactly four cycles.

· cnt goes …8, 9, 0, 1… and never reaches 10.

Deliverable: one annotated GTKWave screenshot per design, with the key transition circled and one sentence explaining it.

Experiments that teach the lesson

1. Change `<=` to `=` in shift4 and re-run. How many flip-flops does Yosys report now?

2. Delete the `if (cnt == 4'd9)` line. Watch the counter run to 15 — a mod-16 counter.

3. Toggle `en` low for three cycles and confirm the count freezes.

HANDS-ON · TUTORIAL T4

The 1011 FSM — Simulate, Then Synthesise It

Goal: close the loop. Take the FSM you designed on paper, run it, then ask Yosys what hardware it actually becomes — and check that the answer matches your state diagram.

Four steps, four commands

```
# ---- Step 1 : simulate  (seq_detect_1011.v is the module from slide 55) ----
iverilog -g2012 -o fsm.out seq_detect_1011.v tb_seq_detect.v
vvp fsm.out                      # testbench drives 1011011 and checks Z pulses twice
gtkwave fsm.vcd &

# ---- Step 2 : synthesise and read the statistics ----
yosys -p 'read_verilog seq_detect_1011.v; \
          synth -top seq_detect_1011; \
          stat'

# ---- Step 3 : look at the schematic Yosys produced ----
yosys -p 'read_verilog seq_detect_1011.v; proc; opt; show -format dot -prefix fsm'
dot -Tpng fsm.dot -o fsm.png     # needs graphviz:  sudo apt install graphviz

# ---- Step 4 : map to a real cell library and count gates ----
yosys -p 'read_verilog seq_detect_1011.v; synth -top seq_detect_1011; \
          abc -g AND,OR,XOR,NAND,NOR; stat; write_verilog fsm_netlist.v'

# ---- or run the whole lab in one go ----
cd Topic3_Lab && ./scripts/run_all.sh && ./scripts/synth_all.sh
```

What the stat report should tell you

· 18 cells in total

· 3 DFF cells — your 3-bit state register

· 15 combinational cells

· ZERO $_DLATCH_ cells

A latch means a default assignment is missing.

Experiments — each gives a real, different answer

1. Synthesise broken_latch.v → find $_DLATCH_ in the cell list.

2. Synthesise seq_detect_1011_onehot.v → 14 cells and 5 flip-flops, not 18 and 3. The only change is one attribute.

3. Run fsm_detect → Yosys refuses to re-encode this FSM because the safe default makes it look self-resetting.

This is the whole point of the topic: *you predicted the hardware from the state diagram, and the tool agreed with you.*

HANDS-ON · ASSESSMENT

Lab Deliverables and Marking Rubric

Deliverable	Evidence required	Marks
T1 — Logisim circuits	half_adder.circ, adder4.circ, fsm1011.circ + one screenshot each	**15**
T2 — Full adder simulation	full_adder.v, tb_full_adder.v, console showing PASS, GTKWave screenshot	**15**
T3 — Sequential designs	dff.v, shift4.v, bcd_counter.v, three annotated waveform screenshots	**20**
T4 — FSM end to end	seq_detect_1011.v, testbench, waveform showing TWO Z pulses, Yosys stat output	**25**
Written answers	Workbook Part 5 exercises E1–E14 with working shown	**15**
Viva / explanation	Explain one design of the examiner's choosing at the whiteboard	**10**

Pass criteria

50 % overall, with a compulsory pass in T4 — a student who cannot take an FSM from state diagram to a verified, synthesised netlist has not met the terminal outcome for this topic. Resubmission of T4 is permitted once.

Key Terms — Boolean Algebra and Combinational Logic

Literal — A variable or its complement — A and A' are two literals.

Minterm / Maxterm — The AND term true for exactly one row / the OR term false for exactly one row.

Canonical form — SOP or POS written directly from the truth table; unique but rarely minimal.

Implicant — Any product term that covers only 1s of the function.

Prime implicant — An implicant that cannot be combined further into a larger group.

Essential prime implicant — The only prime implicant covering some particular minterm — always in the answer.

Don't-care (X) — An input combination that cannot occur, or whose output is never read.

Universal gate — NAND or NOR — enough on its own to build any function.

Duality — Swap AND↔OR and 0↔1; a true Boolean identity stays true.

Fan-in / Fan-out — Number of inputs to a gate / number of loads driven by one output.

Propagation delay t_pd — Time from an input change to the corresponding output change.

Critical path — The slowest input-to-output route; it sets the block's maximum speed.

Hazard / glitch — A transient wrong output caused by unequal path delays; logic is still correct.

Gate equivalent (GE) — Area unit: the area of one 2-input NAND cell in that technology.

Every term above appears in the workbook glossary with a worked micro-example.

Key Terms — Sequential Logic and State Machines

Latch — Level-sensitive storage: transparent for as long as its enable is asserted.

Flip-flop — Edge-triggered storage: samples its input at one instant per clock cycle.

Setup time t_su — How long data must be stable BEFORE the clock edge.

Hold time t_h — How long data must stay stable AFTER the clock edge.

Clock-to-Q t_cq — Delay from the clock edge until the output is valid.

Metastability — An invalid intermediate output state after a setup/hold violation; resolves at random.

Synchroniser — Two (or three) cascaded flip-flops that make metastable failure vanishingly unlikely.

Clock skew — Fixed difference in edge arrival time between flip-flops; caused by the clock tree.

Jitter — Random cycle-to-cycle variation in edge arrival; caused by noise and the PLL.

Slack — Available time minus required time. Positive = met, negative = violation.

State — An equivalence class of input histories — everything the circuit needs to remember.

Moore machine — Output depends on the state only; glitch-free, reacts one cycle later.

Mealy machine — Output depends on state AND input; fewer states, reacts immediately, can glitch.

Safe FSM — One whose default branch returns any illegal state code to a known good state.

Acronyms: SOP sum of products · POS product of sums · FSM finite state machine · CDC clock-domain crossing · STA static timing analysis · CTS clock-tree synthesis · MTBF mean time between failures.

TOPIC 3 · RECAP

Consolidation and Self-Check

3a — Boolean algebra and gates

· Digital = two voltage bands with a guard band between them.

· Three operators; seven gates; NAND and NOR are universal.

· De Morgan lets you push bubbles through any network.

· Truth table → canonical SOP/POS → K-map → minimal form.

· Minimisation buys area, power and speed — all three at once.

3b — Combinational logic

· No feedback, no memory; output = f(inputs now).

· Seven-step procedure; the truth table is where bugs hide.

· Adders, MUX, decoders, comparators, ALU — know all five.

· Ripple carry is O(n); lookahead is O(log n).

· Hazards are real; synchronous design tolerates them.

3c — Sequential logic and FSMs

· Feedback through a clocked element = memory = state.

· Latch is level-sensitive; flip-flop is edge-triggered. Use flip-flops.

· $T \geq t_{cq} + t_{logic} + t_{setup}$ sets f_{max}; hold is a separate race.

· Synchronise every asynchronous input. Always.

· FSM = state register + next-state logic + output logic. Three always blocks.

Self-check — you should be able to answer all ten without notes

1. Prove (A+B)' = A'B' with a truth table. 2. Minimise Σm(0,2,5,7,8,10,13,15) on a 4-variable K-map. 3. Why is NAND universal but AND is not?

4. Draw a full adder from two half adders. 5. A 16-bit ripple adder has t_carry = 80 ps. What is its worst-case delay? 6. Name three hazards and one cure for each.

7. Why does slowing the clock never fix a hold violation? 8. Sketch the 2-flop synchroniser and say what it protects against. 9. Convert the 1011 Moore machine to Mealy — how many states now?

10. Given t_cq = 50, t_logic = 300, t_setup = 40, skew = 20 ps, what is f_{max}?

TOPIC 3 · WHAT'S NEXT

From Logic Diagrams to Verilog — Topic 4

You now have the vocabulary. Topic 4 (RTL Design Using HDL, 6 hours) is where you stop drawing gates and start writing them. Every construct you meet there maps onto something from this topic — which is why this topic came first.

What you learned in Topic 3	How it appears in Topic 4
Truth table, SOP, K-map	assign and always @(*) with case/if
Multiplexer	the ternary operator and every case statement
Full adder, ripple carry	the + operator — and the timing report that follows it
D flip-flop, register	always @(posedge clk) with non-blocking assignment
Reset behaviour	the two reset templates, and why the async one is in the sensitivity list
State diagram, state table	the three-block FSM template with localparam states
Hazard, latch inference	the synthesis warnings you must never ignore

Before the next session

Finish tutorials T1–T4 and the workbook exercises. Bring your working seq_detect_1011.v — Topic 4 begins by rewriting it three different ways and comparing the synthesis results.